

2026 edition

Deep Learning for Music Analysis and Generation

Symbolic Music Generation

($x \rightarrow$ symbolic music)



Yi-Hsuan Yang Ph.D.

yhyangtw@ntu.edu.tw

Reference: Four ISMIR Tutorials

- **“Machine-Learning for Symbolic Music Generation”** by Pierre Roy (Spotify) and Jean-Pierre Briot (Sony CSL) at **ISMIR 2017**
<https://ismir2017.ismir.net/tutorials/index.html#T4>
- **“Generating Music with GANs—An Overview and Case Studies”** by Hao-Wen Dong and Yi-Hsuan Yang (Academia Sinica) at **ISMIR 2019**
<https://salu133445.github.io/ismir2019tutorial/>
- **“Designing Generative Models for Interactive Co-creation”** by Anna Huang (Google), Jon Gillick (UC Berkeley), Chris Donahue (Stanford) at **ISMIR 2021**
<https://github.com/chrisdonahue/music-cocreation-tutorial>
- **“Transformer-based Symbolic Music Generation”** by Berker Banar (QMUL), Pedro Sarmiento (QMUL), Sara Adkins (infinitealbum) at **ISMIR 2023**
<https://ismir2023.ismir.net/tutorials/>

Reference: ACM Multimedia 2021 Tutorial

<https://www.microsoft.com/en-us/research/uploads/prod/2021/10/Tutorial-on-AI-Music-Composition-@ACM-MM-2021.pdf>

A Tutorial on AI Music Composition

Xu Tan & Xiaobing Li

Microsoft Research Asia & Central Conservatory of Music, China

Outline

- **Text-based approach for symbolic music generation**
 - RNN
 - Transformer
- Advanced models for MIDI generation
- Image-based approach for symbolic music generation
- Symbolic representations for music: Beyond MIDI

Two Main Approaches

(2) Consider music as **text**

- Use “tokens” to represents “events” in music
- Becomes a “natural language generation” (NLG) problem

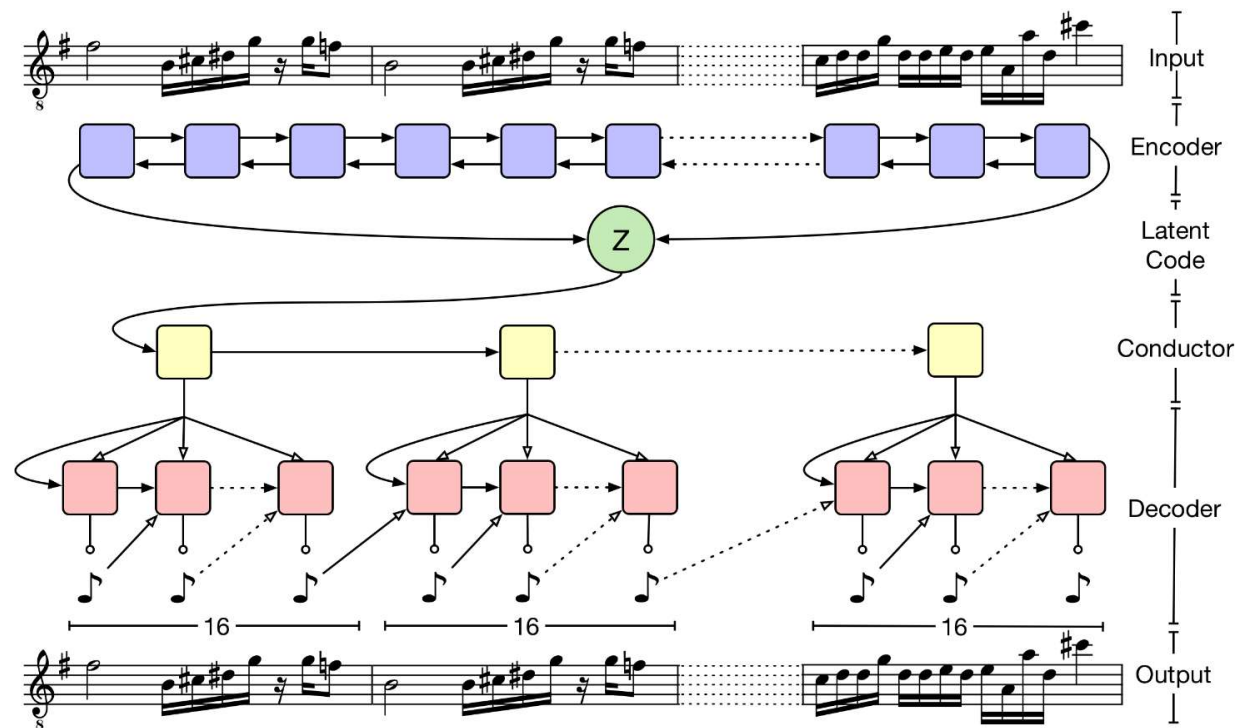
RNN-based Symbolic Music Generation

- Used to be the state-of-the-art
- Good for not-very-long sequences (e.g., 16 bars)
- Examples
 - **MelodyRNN** (2016): <https://magenta.tensorflow.org/2016/06/10/recurrent-neural-network-generation-tutorial>
 - **C-RNN-GAN** (2016)
 - **Song From PI** (2017)
 - **DeepBach** (ICML 2017)

RNN Example: Music VAE

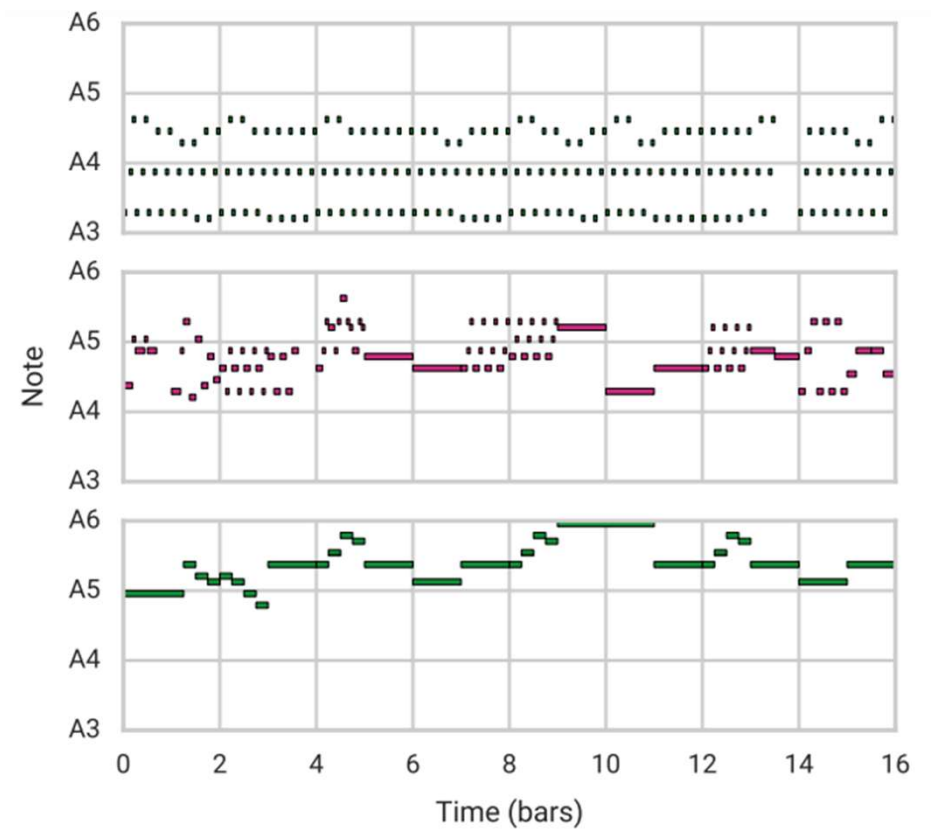
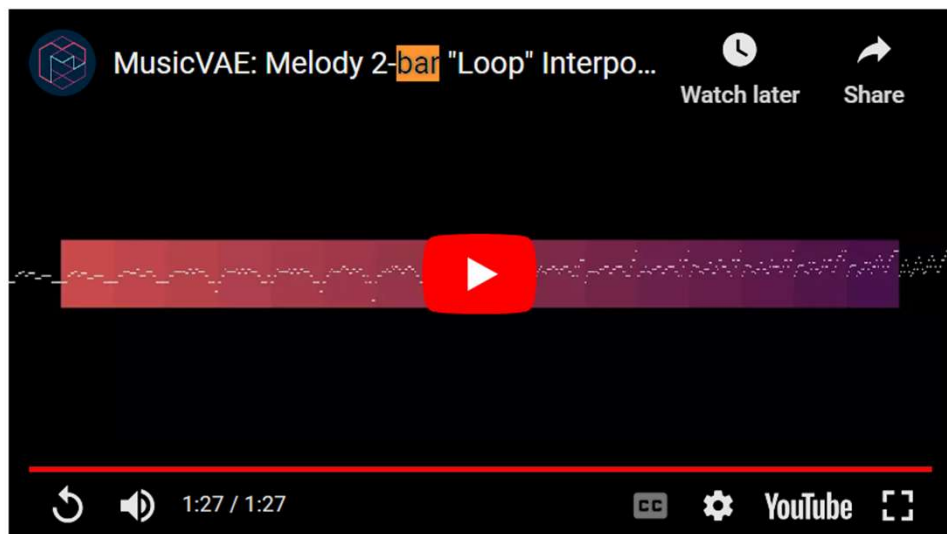
<https://magenta.tensorflow.org/music-vae>

- For **melody** generation
- **Bidirectional encoder**
- **Hierarchical decoder**
 - Bar-level conductor
 - Note-level decoder
- **Recurrent VAE**
 - LSTM for the encoder, conductor, decoder
 - Spherical Gaussian prior for the latent code z



Music VAE

- Latent space manipulation
 - Attribute vectors
 - **Spherical interpolation**



Outline

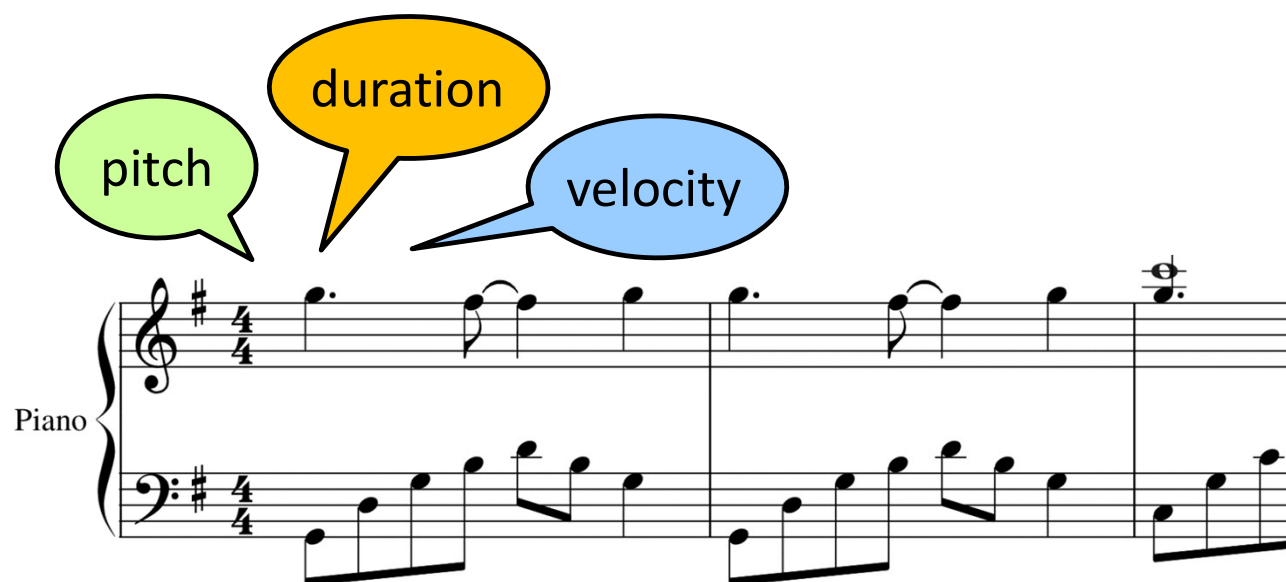
- **Text-based approach for symbolic music generation**
 - RNN
 - **Transformer**
- Advanced models for MIDI generation
- Image-based approach for symbolic music generation
- Symbolic representations for music: Beyond MIDI

Key to the Text-based Approach

- **Neural sequence model**
 - RNNs, hierarchical RNN, Transformers, state-space models (Mamba), H-Net
- **Token representation of music**
 - Token design

Token Design Issue 1: Long Sequence Length

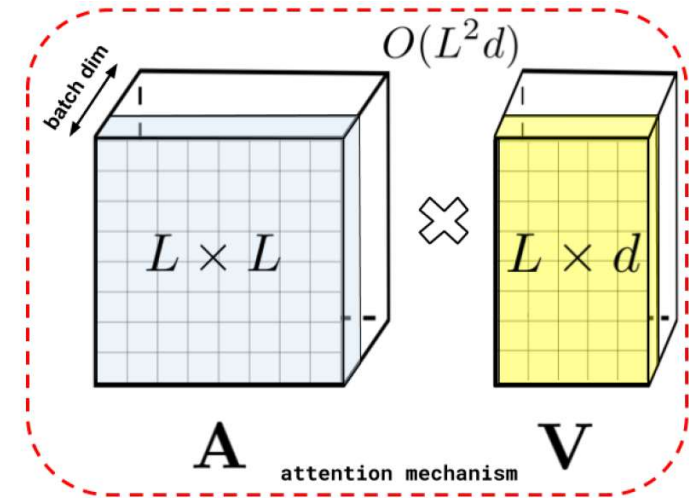
- Both MIDI-like and REMI use multiple tokens to represent a musical note; isn't that inefficient?
- Can we “merge” them without increasing the vocabulary size?



The Problem: Transformers are Expensive

- **Transformers**

- multiple latent vectors
- $y_t = f(y_{t-1}, [h_{t-1}, h_{t-2}, \dots, h_{t-L}])$
- Memory complexity: $O(L^2)$, where L denotes the length of the attention window (i.e., the history) because of the $L \times L$ similarity matrix
- Need to cut a music pieces into **segments** to fit the VRAM



(Figure from the Performer paper)

The Problem: Transformers are Expensive

	Representation	Model	Attn. window
Music Transformer (Huang et al. 2019)	MIDI-like	Transformer	2,048
MuseNet (Payne 2019)	MIDI-like*	Transformer	4,096
LakhNES (Donahue et al. 2019)	MIDI-like*	Transformer-XL	512
(Choi et al. 2020)	MIDI-like	Transformer	2,048
Pop Music T. (Huang and Yang 2020)	REMI	Transformer-XL	512
Transformer VAE (Jiang et al. 2020)	MIDI-like	Transformer	128
Guitar Transformer (Chen et al. 2020)	REMI*	Transformer-XL	512
Jazz Transformer (Wu and Yang 2020)	REMI*	Transformer-XL	512
MMM (Ens and Pasquier 2020)	MIDI-like*	Transformer	2,048

- Most people use **512-token** attention window before 2020
 - But, a piano cover of a pop song can contain up to **5k** tokens
 - **Segmentation** of music pieces makes it hard to learn **long-term patterns**

Compound Word Transformer

<https://github.com/YatingMusic/compound-word-transformer>

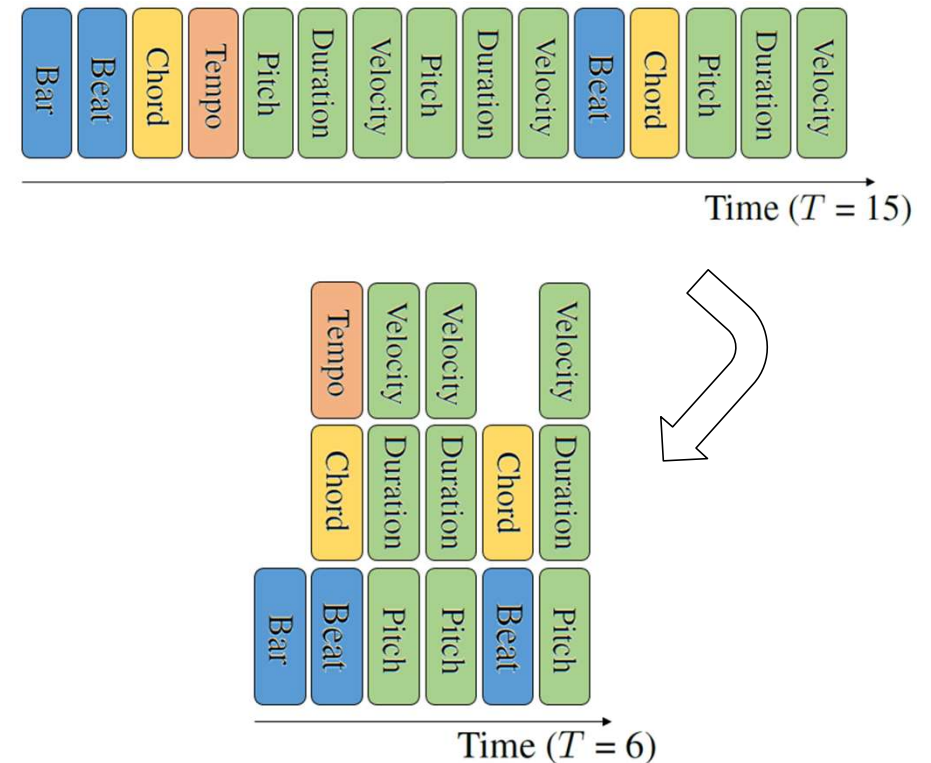
	GPU memory	Training time (single GPU)	Inference time (on GPU)	Quality MOS
Transformer-XL	4-17 GB	3-7 days	2x real time	3.0- 3.3
CP Transformer (ours)	4 GB	1 day	8x real time	3.3

- **Compound Word**: grouping of tokens
- Shorter training time → easier to try different ideas
- Shorter inference time → good for real-time applications

Ref: Hsiao et al, "Compound Word Transformer: Learning to compose full-song music over dynamic directed hypergraphs," AAAI 2021

Compound Word Transformer

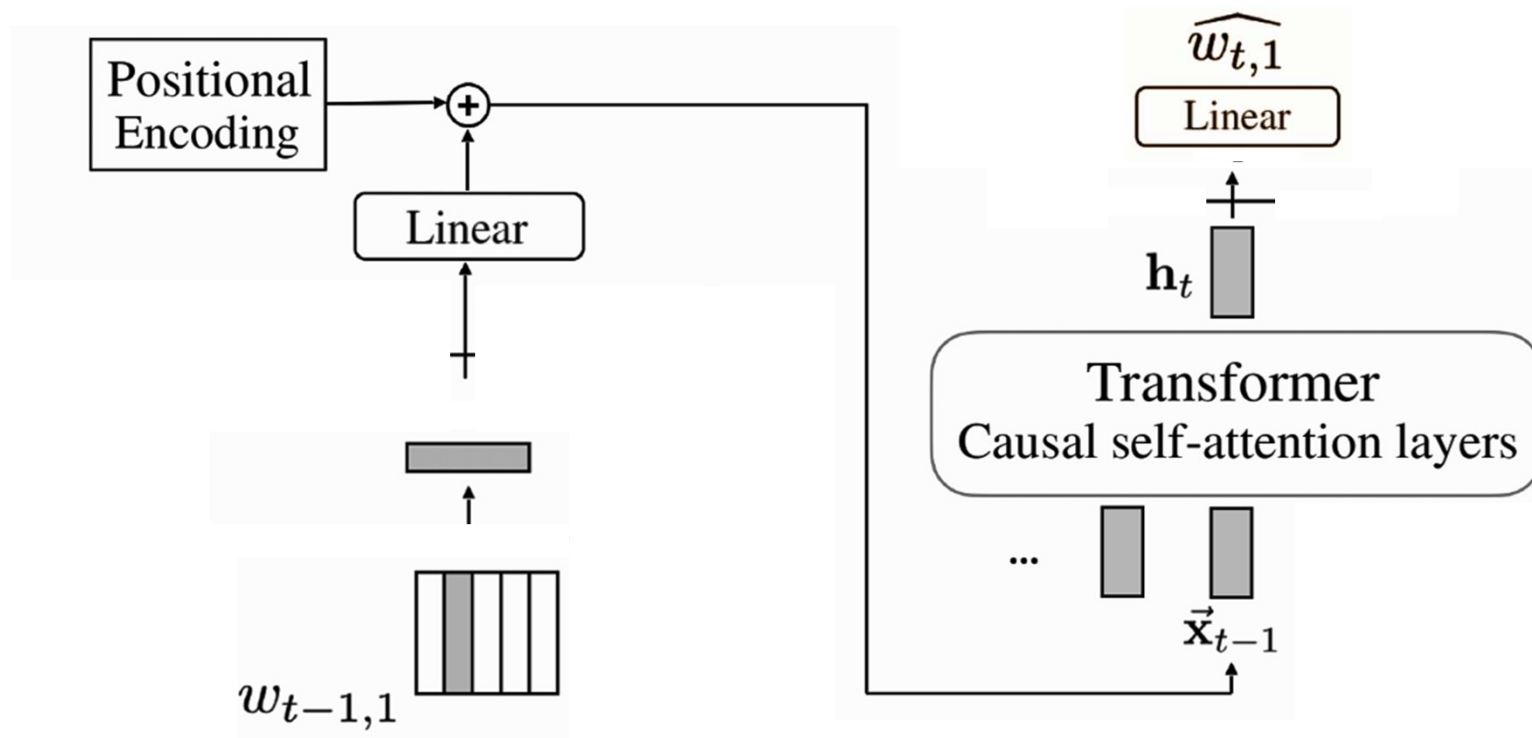
- Unlike the case in text, the vocabulary of music involves tokens of various token types (e.g., note-related, metric-related)
- Therefore, we can
 - group tokens** into “compound words” (e.g., pitch + duration + velocity)
 - predict multiple tokens** of various types **at once in a single time step**
 - the embeddings of the predicted tokens are combined to be used as input to the next time step



Ref: Hsiao et al, “Compound Word Transformer: Learning to compose full-song music over dynamic directed hypergraphs,” AAAI 2021

Original Transformer

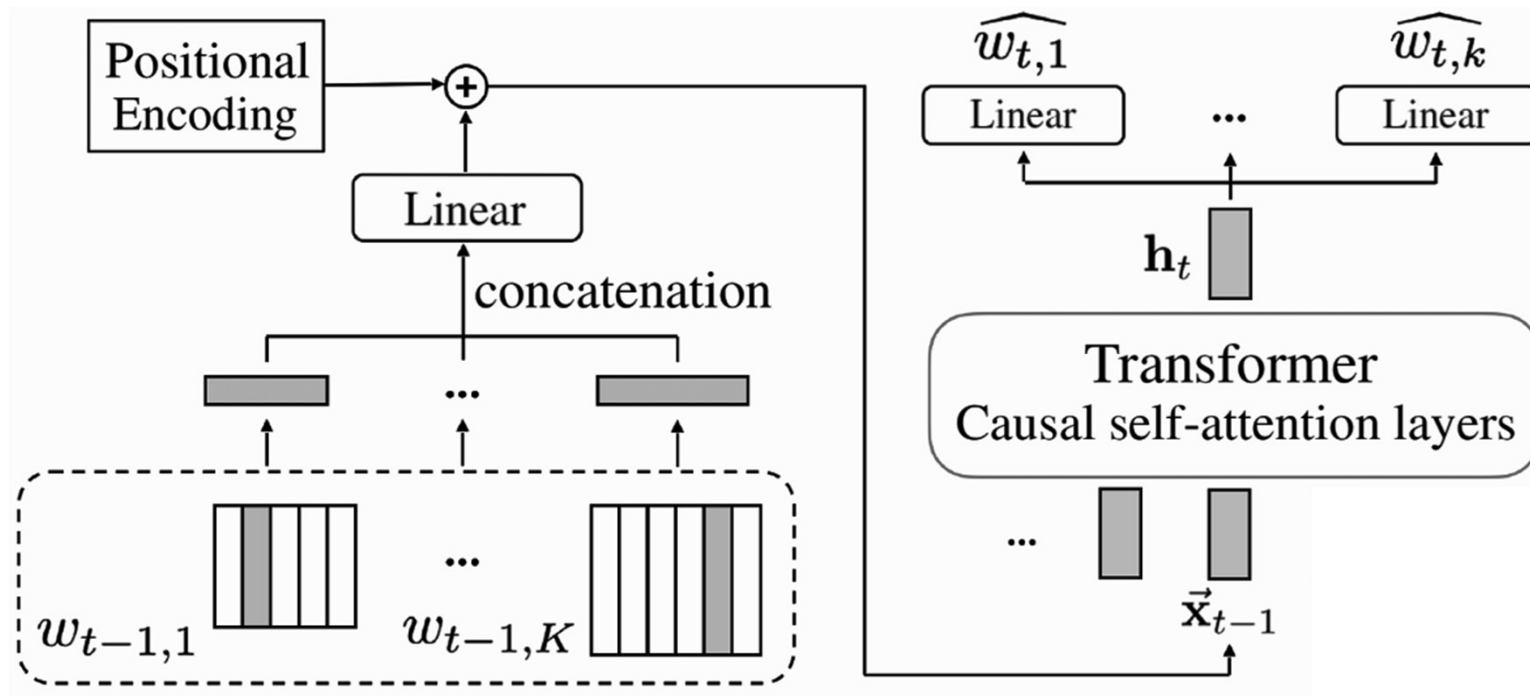
single output at each time step



single input at each time step

Compound Word Transformer

multiple output at each time step



multiple input at each time step

Transformer vs. CP Transformer

- **Transformer**

- $y_t = f(y_{t-1}, [h_{t-1}, h_{t-2}, \dots, h_{t-L}])$
- attention window: **L tokens**

- **CP Transformer**

- multiple output: $cp_t = [y_t^{(1)}, y_t^{(2)}, \dots, y_t^{(K)}]$
- multiple input: $cp_{t-1} = [y_{t-1}^{(1)}, y_{t-1}^{(2)}, \dots, y_{t-1}^{(K)}]$
- $cp_t = f(cp_{t-1}, [h_{t-1}, h_{t-2}, \dots, h_{t-L}])$
- `cp` is a super token
- The latent vectors `h` are associated with each `cp`
- Attention window: **L super-tokens**, or LK tokens

Compound Word Transformer

- We can now feed a **whole song** (up to 10,240 tokens) to our Transformer decoder for training on a single GPU with **11GB** VRAM
- The model converges within 1.5 days using an NVIDIA RTX 2080 Ti

Task	Representation + model@loss	Training time	GPU memory	Inference (/song)	
				time (sec)	tokens (#)
Conditional	Training data	—	—	—	—
	Training data (randomized)	—	—	—	—
	REMI + XL@0.44	3 days	4 GB	88.4	4,782
	REMI + XL@0.27	7 days	4 GB	91.5	4,890
	REMI + linear@0.50	3 days	17 GB	48.9	4,327
	CP + linear@0.27	0.6 days	10 GB	29.2	18,200
Unconditional	REMI + XL@0.50	3 days	4 GB	139.9	7,680
	CP + linear@0.25	1.3 days	4 GB	19.8	9,546

Ref: Hsiao et al, “Compound Word Transformer: Learning to compose full-song music over dynamic directed hypergraphs,” AAAI 2021

Compound Word Transformer

- **Linear Transformer: $O(L)$**
→ save GPU space
- **CP**: shorter sequence length
→ save GPU space further, and converge much faster

Representation + model@loss	Training time	GPU memory
Training data	—	—
Training data (randomized)	—	—
REMI + XL@0.44	3 days	4 GB
REMI + XL@0.27	7 days	4 GB
REMI + linear@0.50	3 days	17 GB
CP + linear@0.27	0.6 days	10 GB

Ref: Katharopoulos et al, “Transformers are RNNs: Fast autoregressive Transformers with linear attention”, ICML 2020

Customed Design for Different Token Types

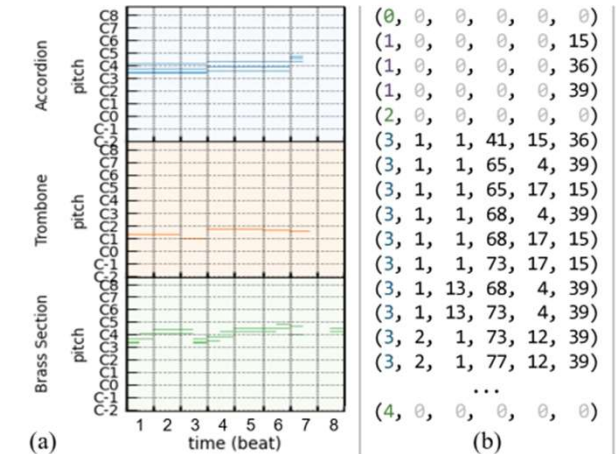
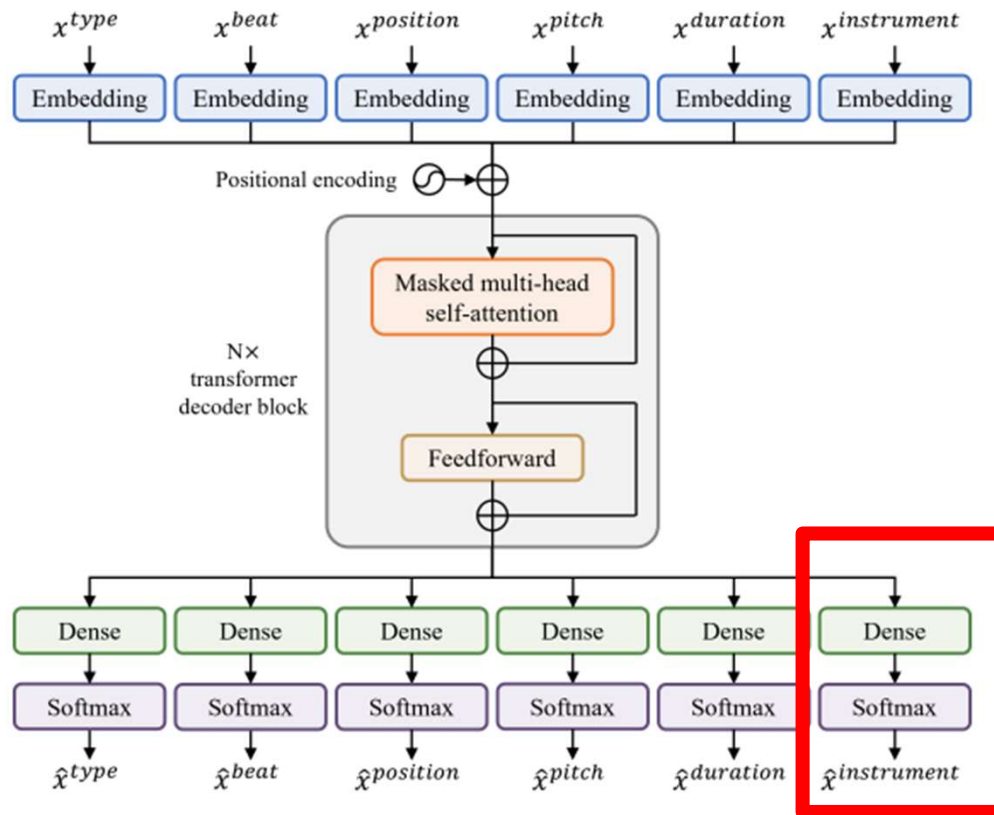
Different embedding size & sampling policy

Repre.	Token type	Voc. size $ \mathcal{V}_k $	Embed. size (d_k)	Sample _k ($\cdot$) τ ρ	
CP	[track]	2 (+1)	3	1.0	0.90
	[tempo]	58 (+2)	128	1.2	0.90
	[position/bar]	17 (+1)	32	1.2	0.90
	[chord]	133 (+2)	256	1.0	0.90
	[pitch]	86 (+1)	512	1.0	0.90
	[duration]	17 (+1)	128	2.0	0.90
	[velocity]	24 (+1)	128	5.0	1.00
	[family]	4	32	1.0	0.99
	total	341 (+9)	—	—	—
REMI	total	338	512	1.2	0.90

Table 3: Details of the CP representation in our implementation, including that of the sampling policy (τ -tempered top- ρ sampling). For the vocabulary size, the values in the parentheses denote the number of special tokens such as [ignore].

Multitrack Music Transformer

<https://hermandong.com/mmt/>



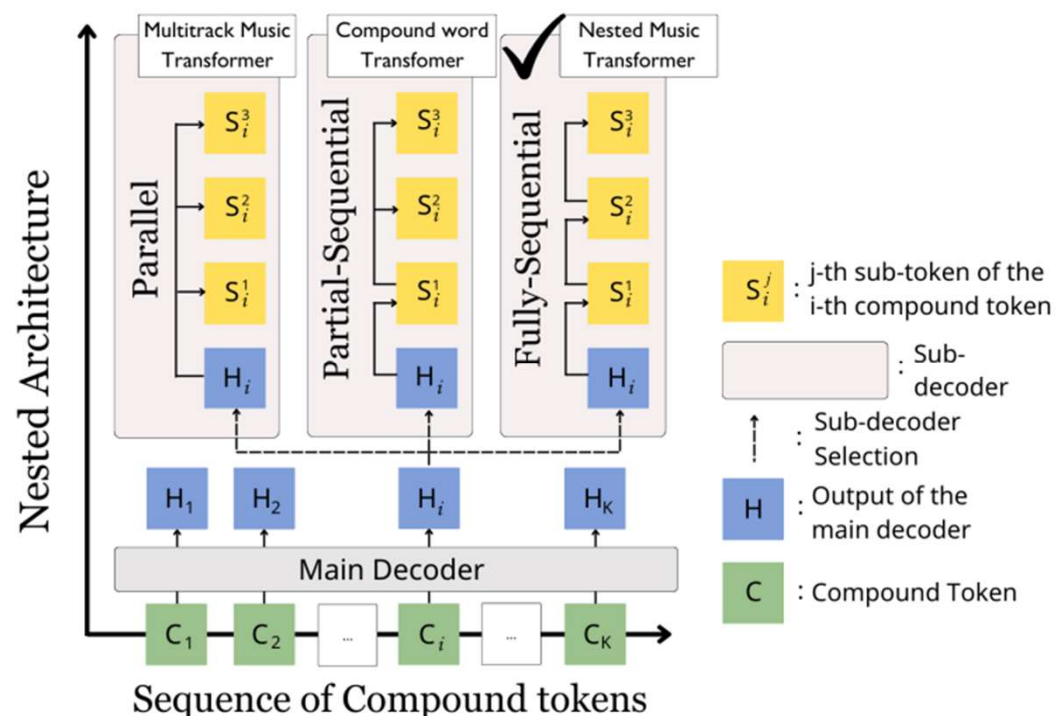
Start of song
Instrument: accordion
Instrument: trombone
Instrument: brasses
Start of notes
Note: beat=1, position=1, pitch=E2, duration=48, instrument=trombone
Note: beat=1, position=1, pitch=E4, duration=12, instrument=brasses
Note: beat=1, position=1, pitch=E4, duration=72, instrument=accordion
Note: beat=1, position=1, pitch=G4, duration=12, instrument=brasses
Note: beat=1, position=1, pitch=G4, duration=72, instrument=accordion
Note: beat=1, position=1, pitch=C5, duration=1, instrument=brasses
Note: beat=1, position=13, pitch=G4, duration=12, instrument=brasses
Note: beat=1, position=13, pitch=C5, duration=12, instrument=brasses
Note: beat=2, position=1, pitch=C5, duration=36, instrument=brasses
Note: beat=2, position=1, pitch=E5, duration=36, instrument=brasses
...
End of song

Ref: Dong et al, "Multitrack Music Transformer," ICASSP 2023

Nested Music Transformer (NMT)

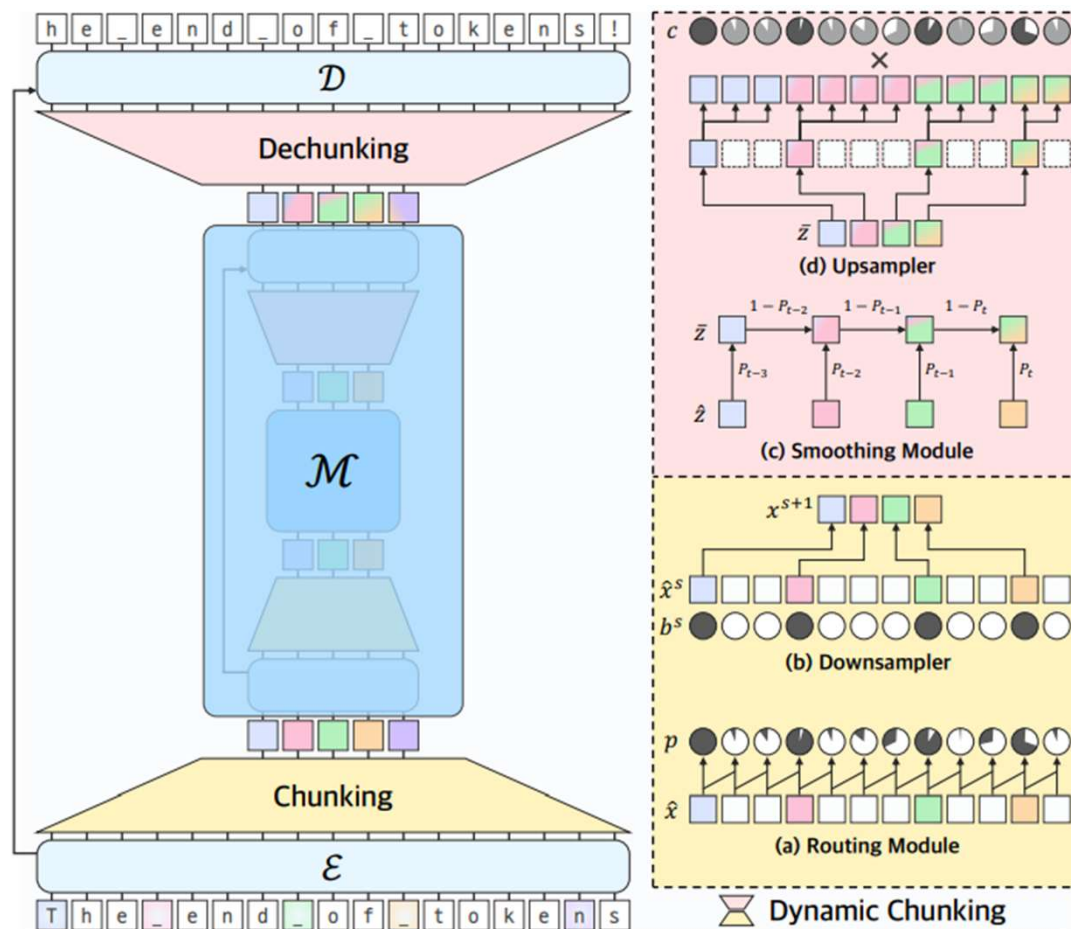
<https://judejiwoo.github.io/nested-music-transformer-demo/>

- Two transformers
 - the main decoder that models a **sequence of compound tokens**
 - the sub-decoder for modeling **sub-tokens of each compound token**
- Slower
- But better model the dependency among the sub-tokens



Ref: Ryu et al, "Nested Music Transformer: Sequentially decoding compound tokens in symbolic music and audio generation," ISMIR 2024

H-Net: Hierarchical Network

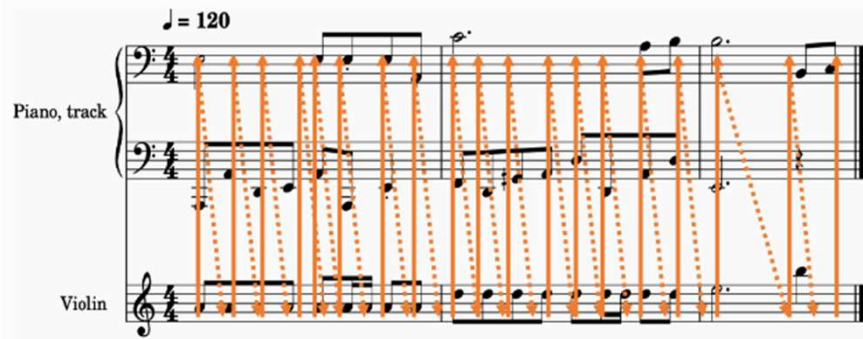


Ref: Hwang et al, "Dynamic chunking for end-to-end hierarchical sequence modeling," ICLR 2026

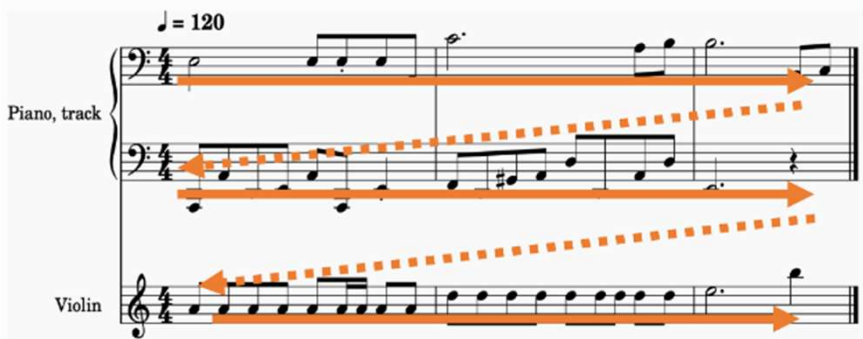
Issue 2: Token Order in Representing Multi-Track MIDI Music

<https://musiclang.github.io/tokenizer/>

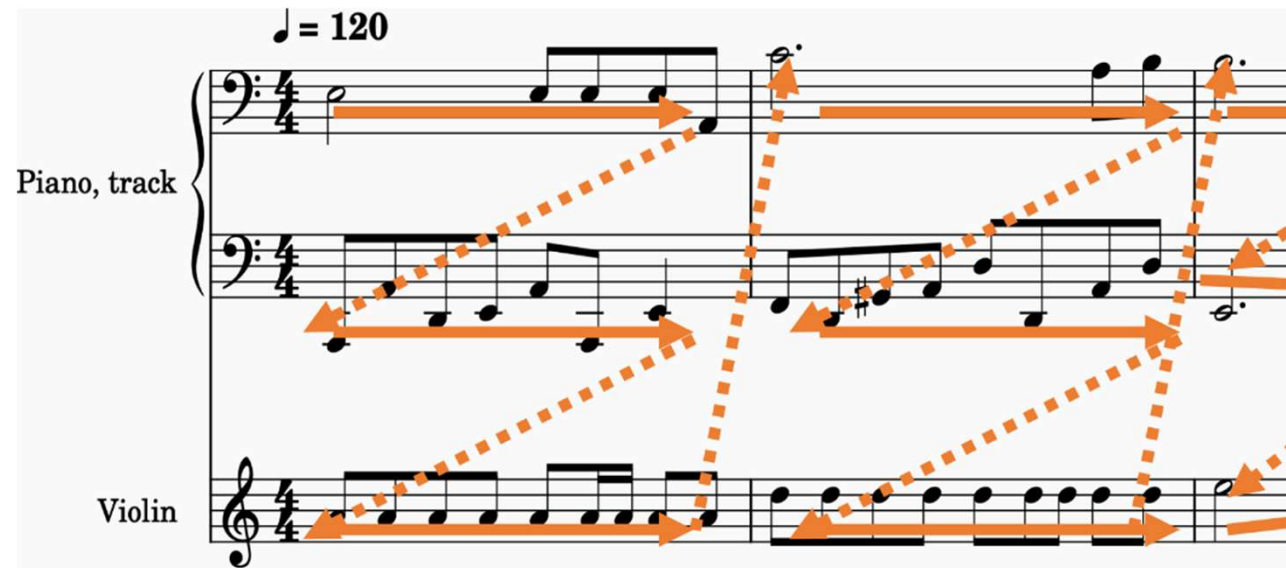
Vertical parsing



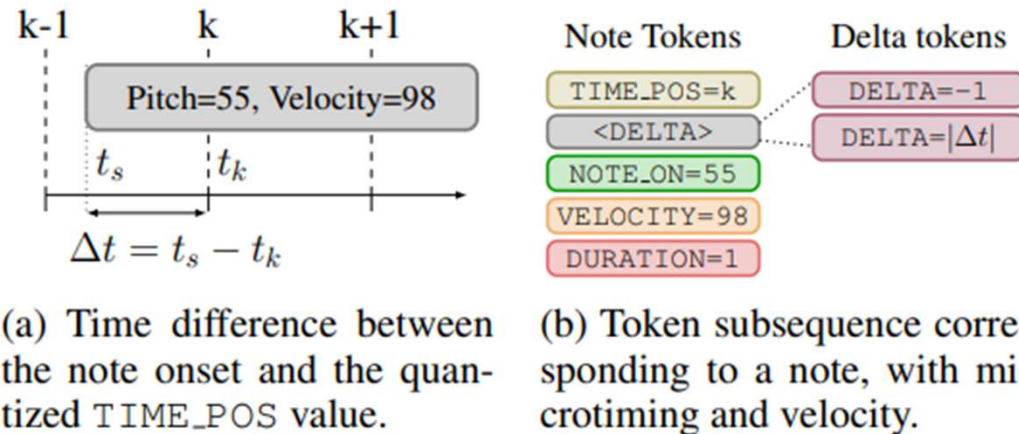
Horizontal parsing



A natural way to process music



Issue 3: Representing Time

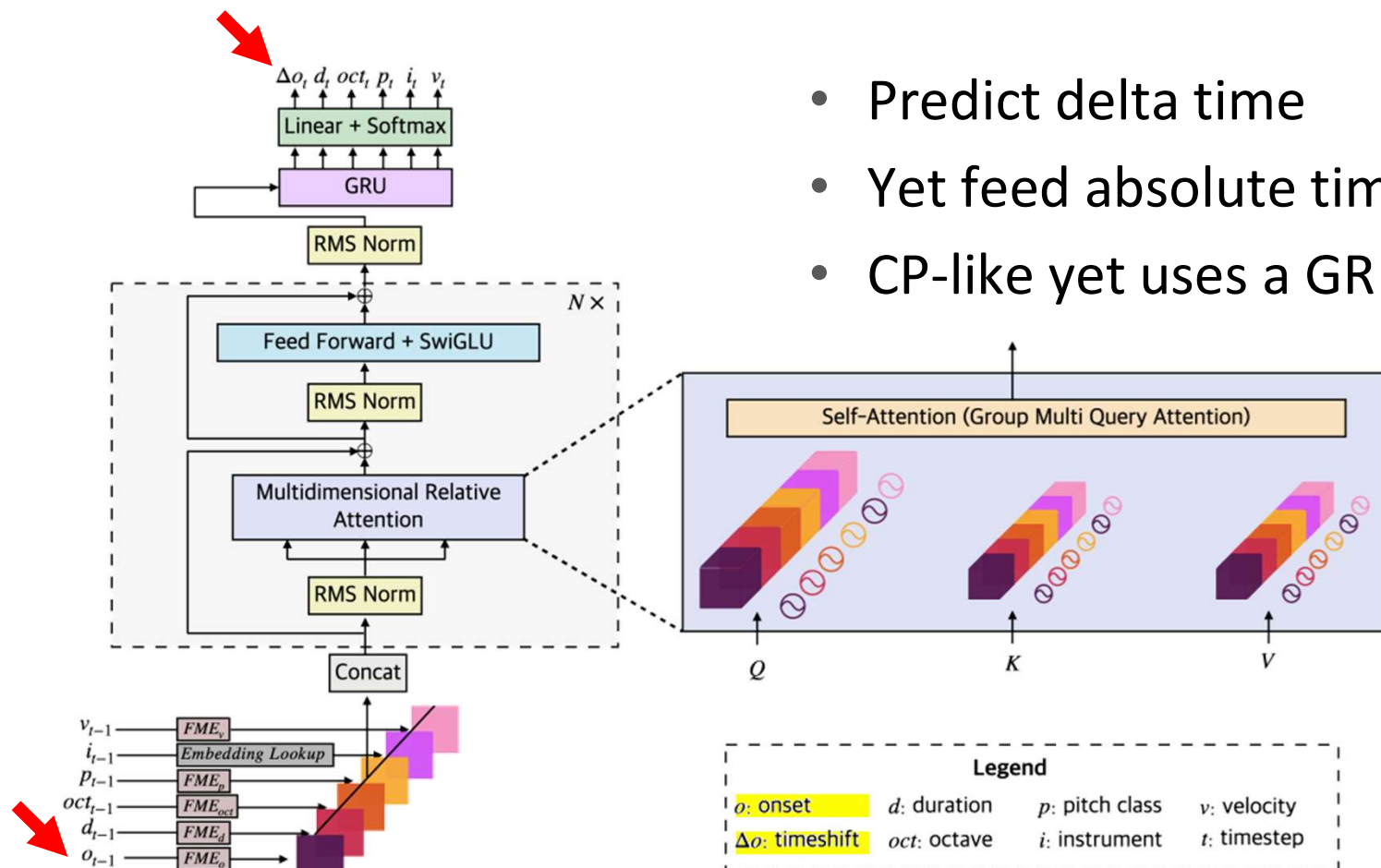


- 96 different TIME POSITION tokens within a bar + **DELTA tokens**
 - DELTA tokens encode the time difference of the original MIDI note onset
 - Relative time + duration
- Add interpretation expressiveness

Ref: Pasquier et al, "MIDI-GPT: A controllable generative model for computer-assisted multitrack music composition," AAAI 2025

Issue 3: Representing Time

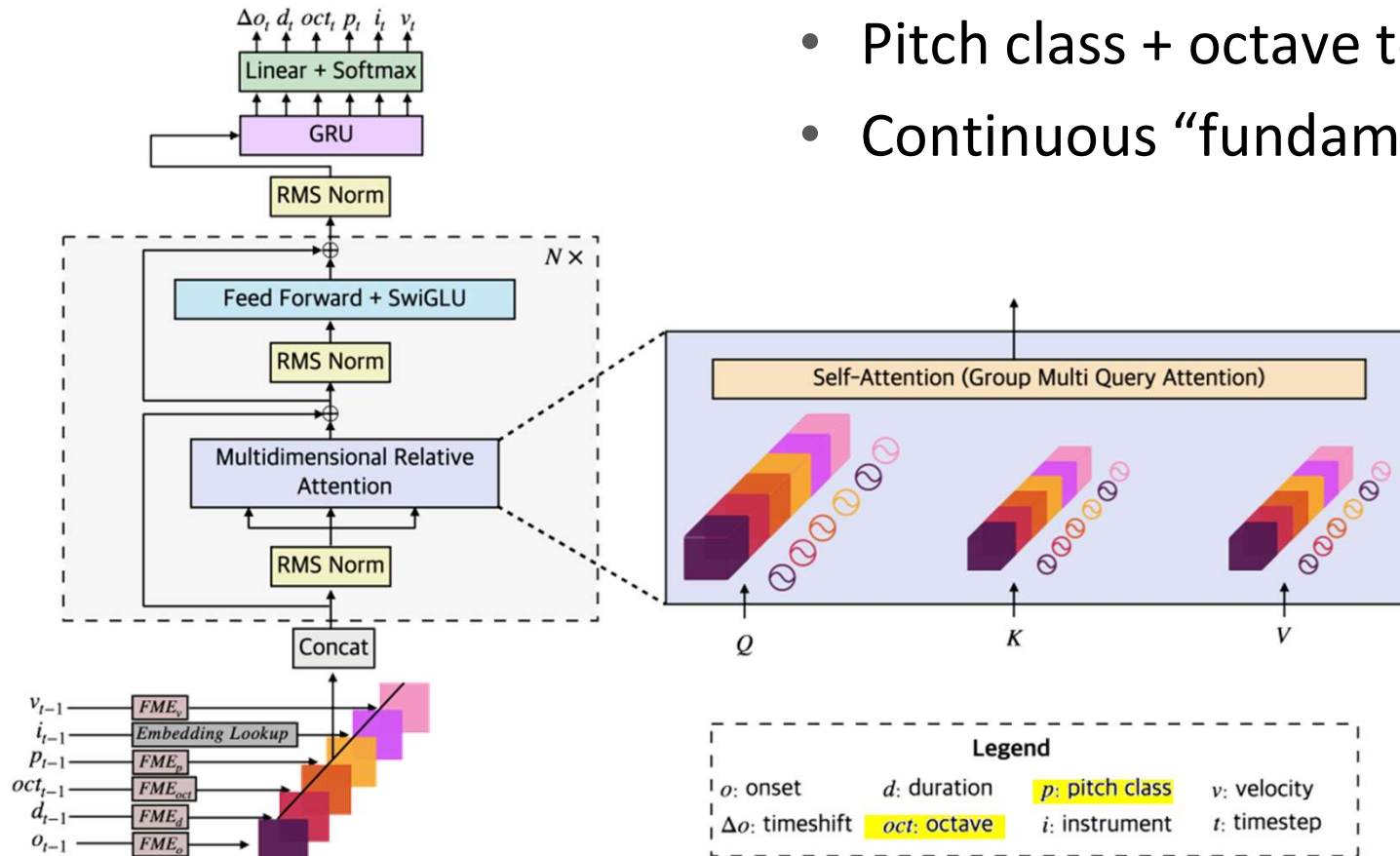
- Predict delta time
- Yet feed absolute time as input
- CP-like yet uses a GRU to predict (NMT)



Ref: Guo & Dixon, "Moonbeam: A MIDI foundation model using both absolute and relative music attributes," arXiv 2025

Issue 4: Representing Pitch

- Pitch class + octave tokens
- Continuous “fundamental music embedding”



Ref: Guo & Dixon, “Moonbeam: A MIDI foundation model using both absolute and relative music attributes,” arXiv 2025

Issue 5: Token Representation for Key

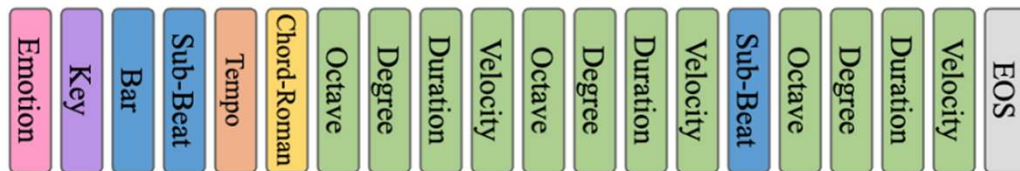
<https://emo-disentangler.github.io/>

- **Functional representation**

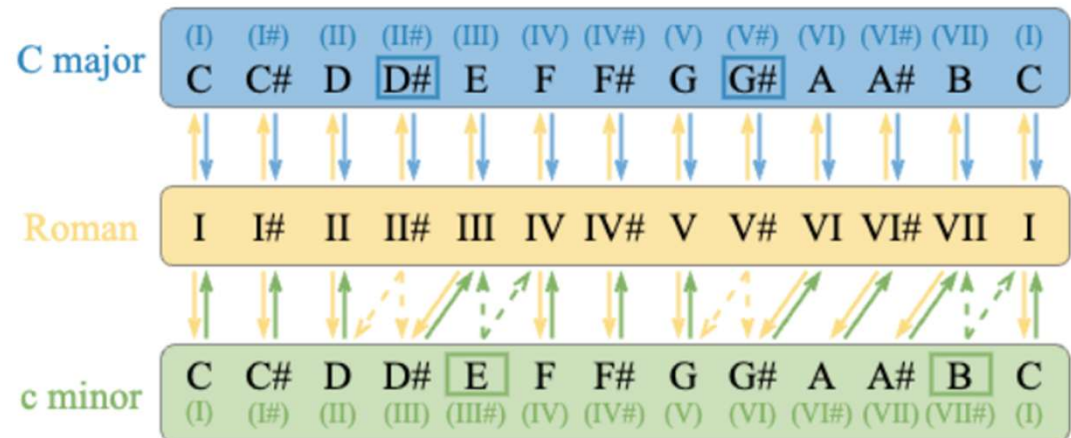
- Encode both melody and chords with Roman numerals relative to musical keys, to consider the interactions among notes, chords and tonalities



(a) REMI representation



(b) Functional representation



Ref: Huang et al, "Emotion-driven piano music generation via two-stage disentanglement and functional representation", ISMIR 2024

Issue 6: Token Representation for Chords

<https://musiclang.github.io/tokenizer/>

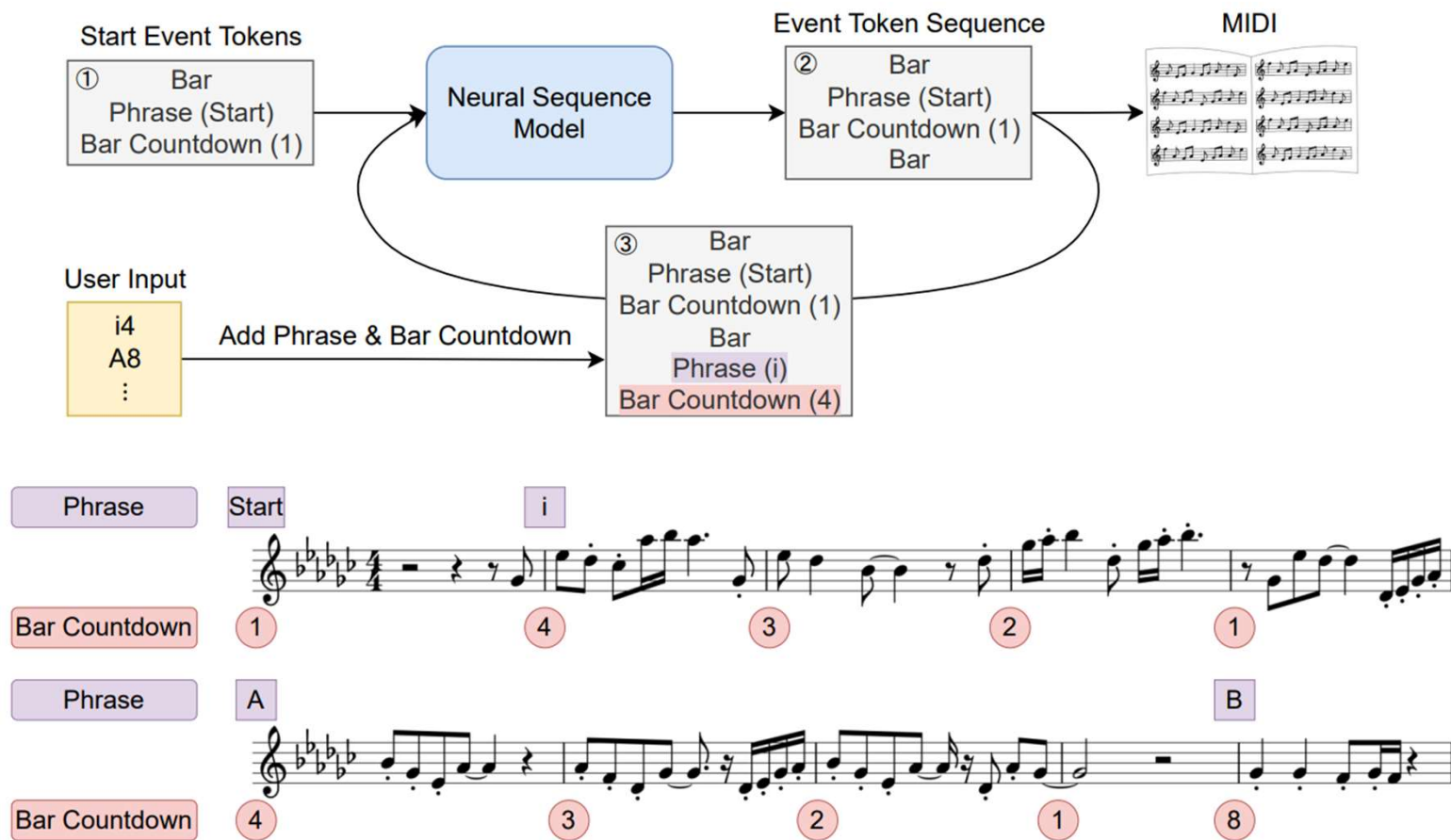
- Root, quality, inversion, etc

Each chord is depicted using seven tokens (six control tokens plus one "CHORD_CHANGE" token):

- **Scale degrees:** I, II, III, IV, V, VI, VII - represent the chord's degree within the scale. This does not specify the chord's type; the mode and extension are needed for that.
- **Tonality root:** 0 to 12, corresponding to each pitch class possible (C to B).
- **Tonality mode:** m for harmonic minor, M for major.
- **Chord octave:** -4 to 4, indicating the root note's octave relative to the 4th octave.
- **Extension:** Uses figured bass notation to detail the chord's bass and any extensions (e.g., " (triad in root position), '6', '64', '7', '65', '43', '2'), including variations with sus2 or sus4 for suspended chords.
- **Duration numerator/denominator:** Numerical values representing the chord's fractional duration, providing control over the time signature (since one chord equals one bar in MusicLang).

Issue 7: Token Representation for Musical Structure

<https://mil-tokyo.github.io/phrase-length-designated-music-generation/>



Outline

- Text-based approach for symbolic music generation
- **Advanced models for MIDI generation**
 - Theme Transformer
 - MuseMorphose and VQVAE-Transformer
 - Compose-and-embellish
- Image-based approach for symbolic music generation
- Symbolic representations for music: Beyond MIDI

Conditional Methods for Transformers

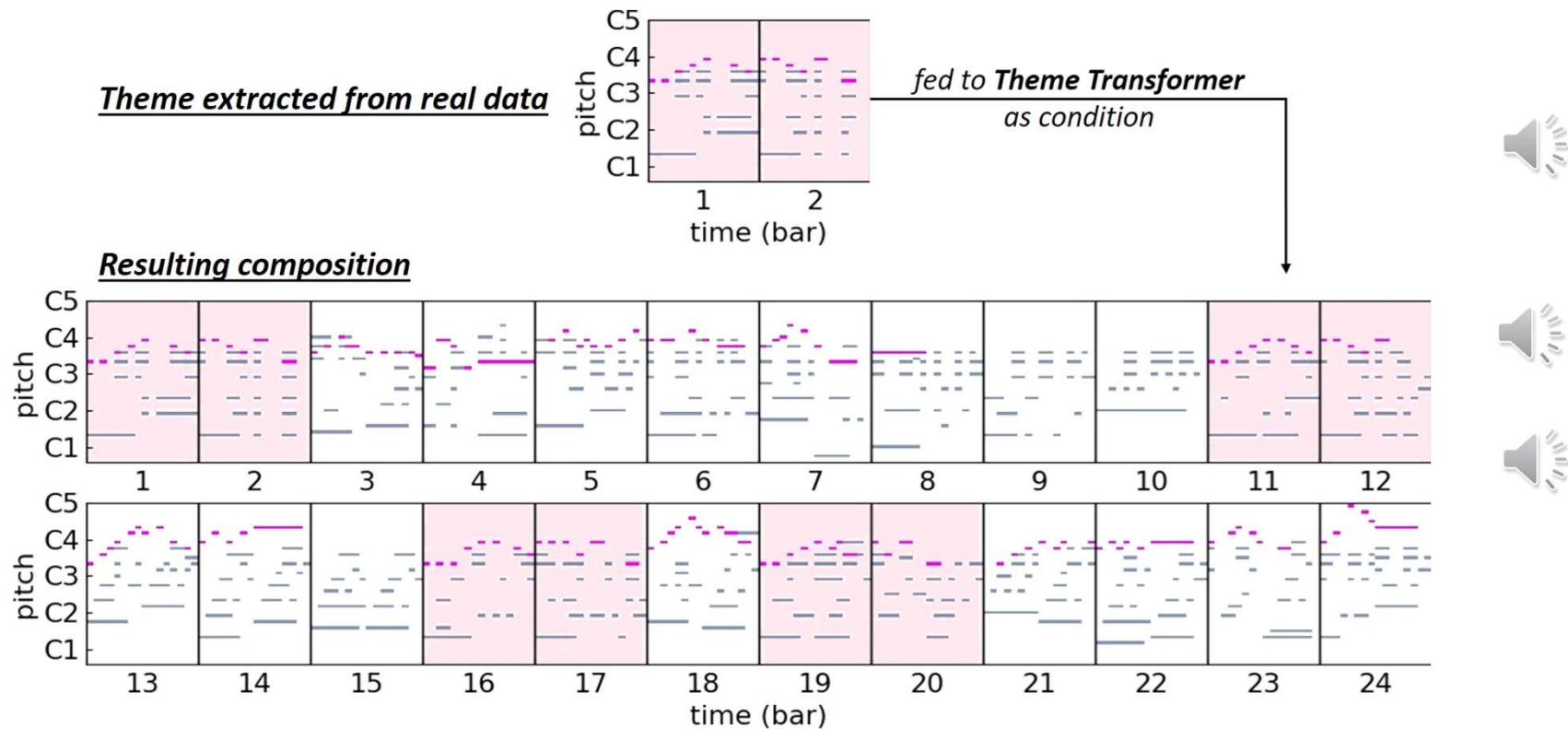
	Condition type	Conditioning mechanism
Theme Transformer	token sequence (theme)	cross attention
MuseMorphose	embedding sequence	in-attention
Compose & Embellish	token sequence (lead sheet)	interleaving prompt

Theme Transformer: Theme-based Conditioning

- Conventional **prompt-based conditioning**
 - Cannot guarantee that the conditioning sequence would repeat or develop in the created continuation
 - Moreover, the influence of the prompt would diminish over time and the generated music *drifts away* from its beginning as the music gets longer
- Proposal: **theme-based conditioning**
 - **Automatic theme finding**: use fragments that have multiple occurrences in a music piece as the conditioning sequence at training time
 - Use **separate memory network** so that the model learns to **exploit the theme condition** when appropriate

Theme Transformer: Theme-based Generation

<https://atosystem.github.io/ThemeTransformer/>



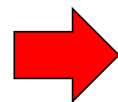
Ref: Shih et al, "Theme Transformer: Symbolic music generation with theme-conditioned Transformer", TMM 2023

Theme Transformer: Automatic Theme Finding

- Use **contrastive learning** to get **melody embedding**
 - **Negative pair**: fragments from different pieces
 - **Positive pair**: to be invariant to the following variations
 - Pitch shift on scale
 - Last note duration variation
 - Note splitting and combination
- Use DBSCAN to find clusters for each piece of music
- Pick the **largest cluster** as the theme for a piece of music

Theme Transformer: Token Design

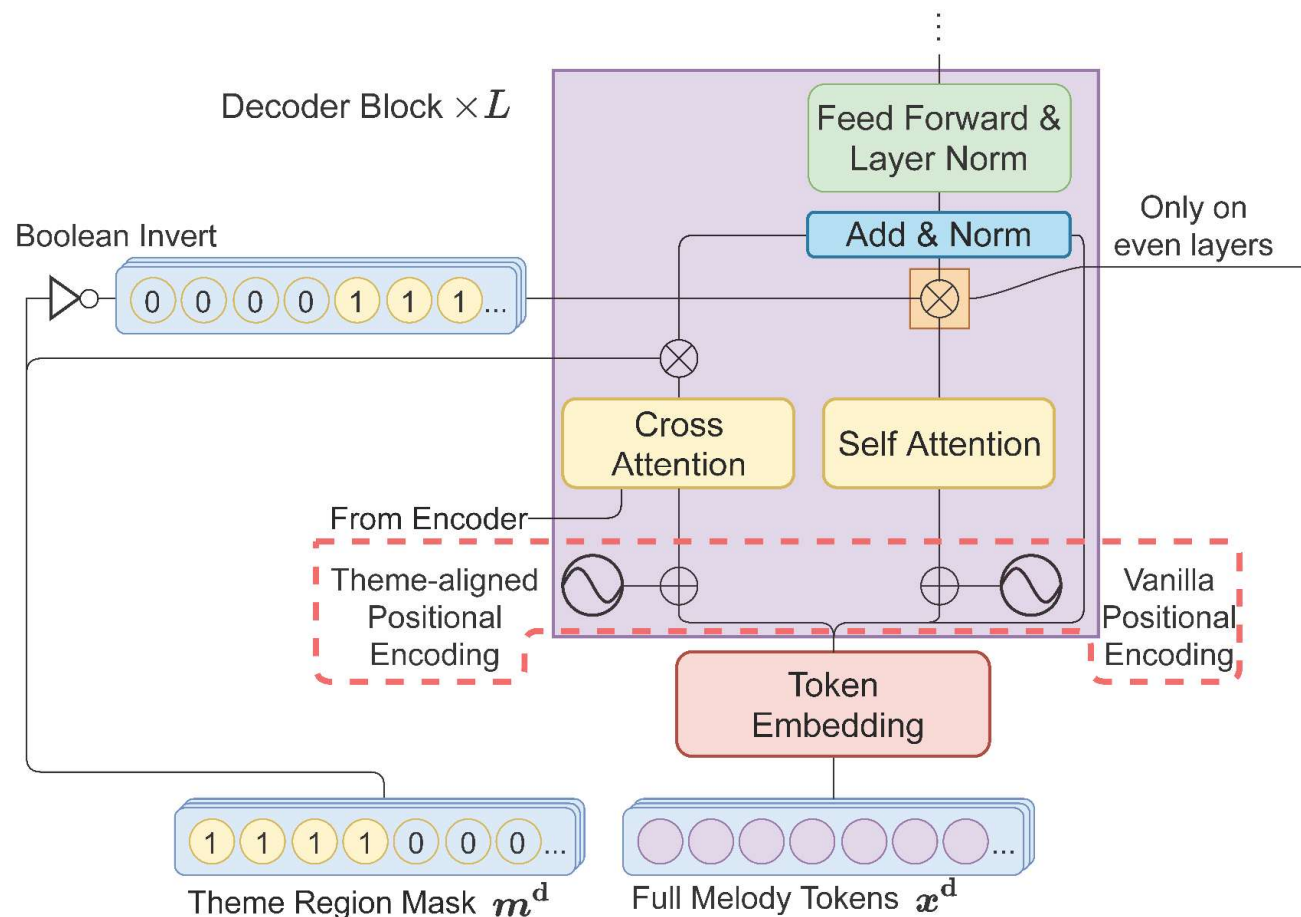
- Use **THEME-START** and **THEME-END** tokens to indicate the beginning and ending of a *theme region*
- At inference time, the model decides on its own when to enter a theme region



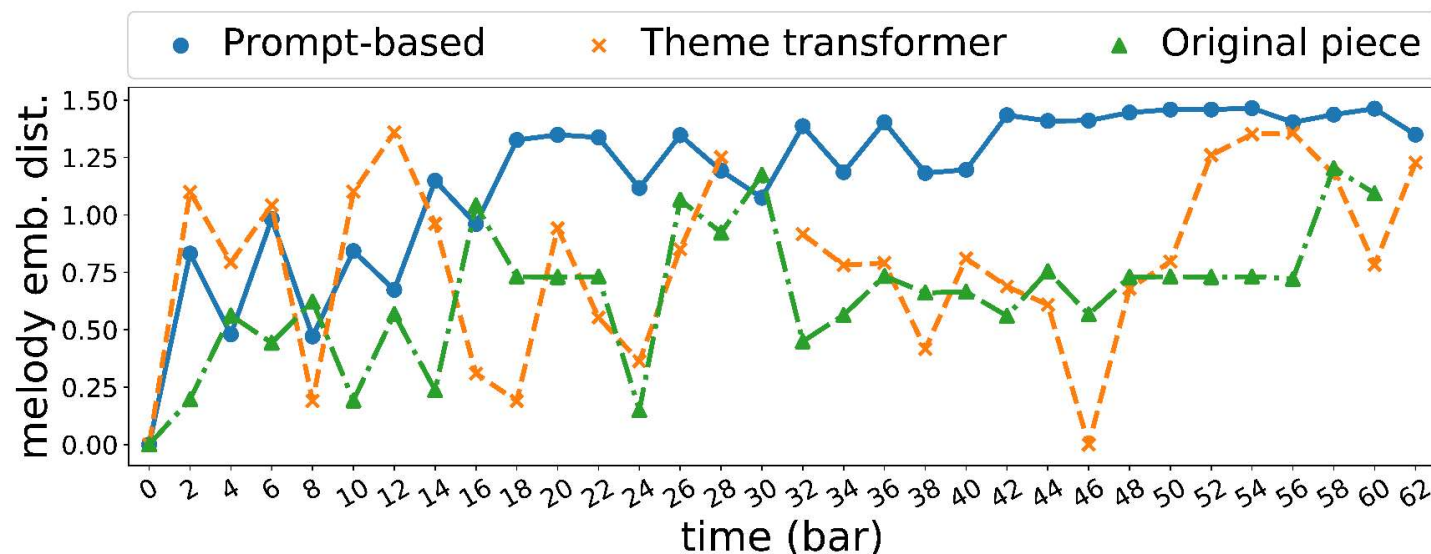
Token type	Piano Representation
NOTE-PITCH	127×2
NOTE-DURATION	64×2
NOTE-VELOCITY	126×2
TEMPO	76
BAR	1
SUBBEAT	16
THEME	2
PADDING	1
Total	730

Theme Transformer: Attention Design

- **Separate memory network**
 - **Self-attention:** to attend to the past (local coherence)
 - **Cross-attention:** to attend to the theme
- **“Switch on” cross-attention only during theme regions**
- Dedicated **theme-aligned positional encoding**



Theme Transformer: Evaluation



	Pitch class consistency↑	Melody inconsistency↓	Grooving consistency↑	Theme inconsistency↓	Theme uncontrollability↓
Baseline (Huang et al. 2019)	.59±.07	.33±.38	.84±.09	—	—
Seq2seq Transformer (proposed)	.61±.04	.46±.28	.90±.06	.22±.03	.45±.19
Theme Transformer (proposed)	.61±.06	.13±.24	.92±.07	.15±.06	.34±.13
Original pieces	.65±.05	.09±.18	.74±.10	.11±.06	.09±.06

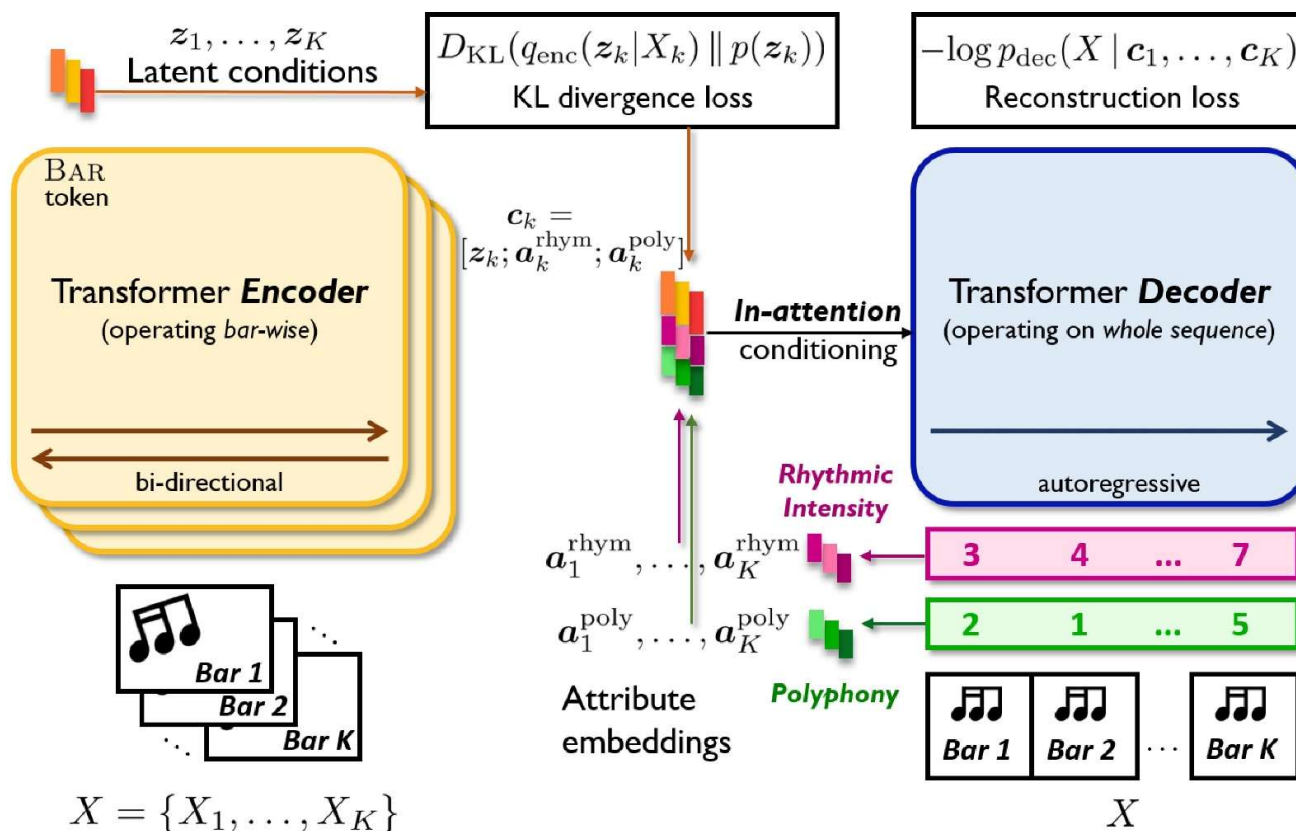
Ref: Shih et al, “Theme Transformer: Symbolic music generation with theme-conditioned Transformer”, TMM 2023

Theme Transformer: Strength and Weakness

- Allow us to repeat something over
- But
 - No development
 - Cannot control the structure
- Question: Can we generate a **skeleton/outline** of the music first, before we fill the details?

MuseMorphose: VAE-Transformer for Style Transfer

<https://slseanwu.github.io/site-musemorphose/>



Ref: Wu & Yang, "MuseMorphose: Full-song and fine-grained piano music style transfer with one Transformer VAE", TASLP 2023

The Condition Might be Ignored!

- Happens when the decoder is powerful, or when the decoding is auto-regressive, where previous tokens in the sequence reveal strong information
 - Reduce the power of the decoder
 - Increase the relevance or understandability of the condition
 - Use a stronger conditioning mechanism
- Size of the information bottleneck
 - Having a narrow enough latent space bottleneck is a key to successful style control
 - When the latent space is unconstrained, as in the case of AE objective, the latent conditions may carry all necessary information for the decoder to reconstruct the music, rendering the other source of conditions, i.e., the attribute embeddings, which do not connect to the encoder, fruitless

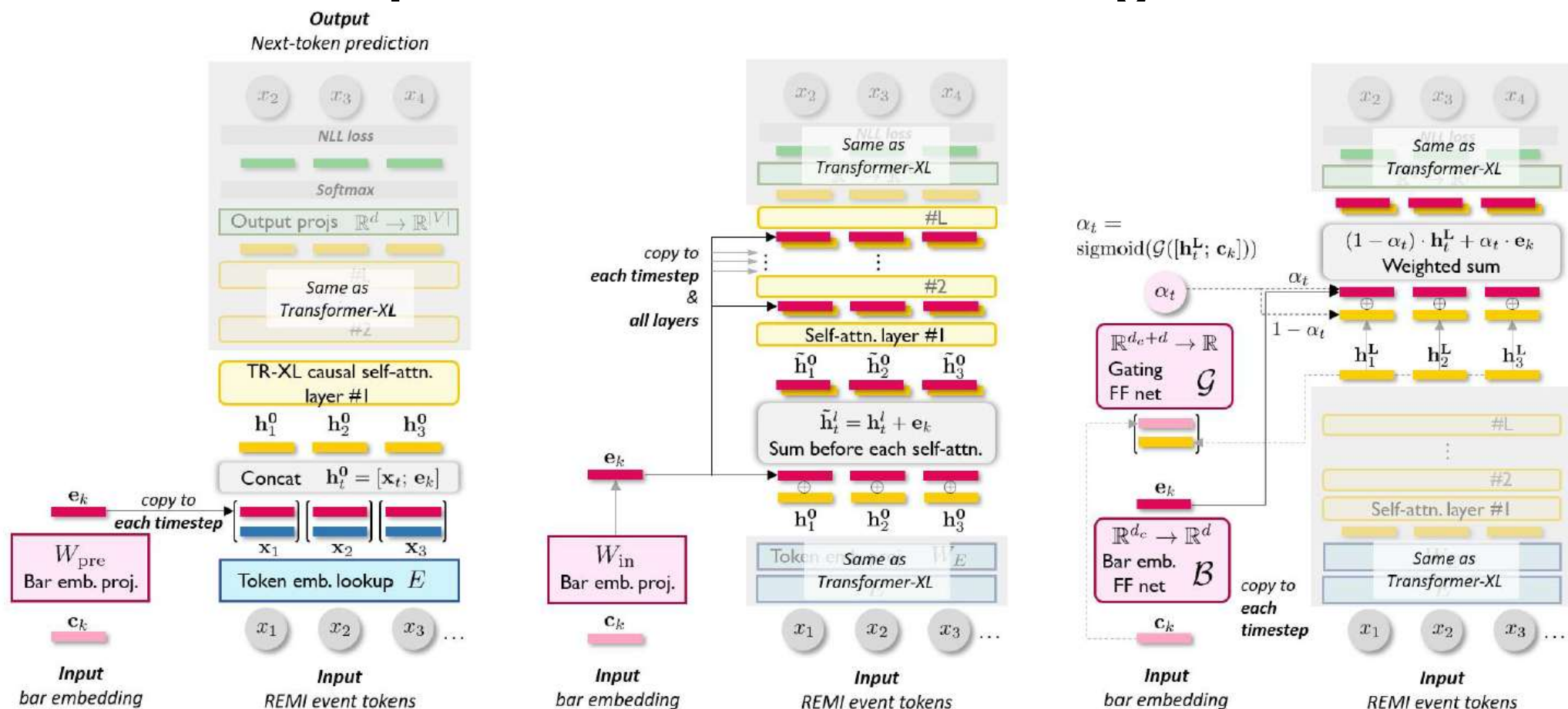
$$D_{\text{KL}}(q_{\text{enc}}(z_k | X_k) \parallel p(z_k))$$

KL divergence loss

$$-\log p_{\text{dec}}(X | c_1, \dots, c_K)$$

Reconstruction loss

MuseMorphose: Various Conditioning Mechanisms



(a) Pre-attention conditioning

(b) In-attention conditioning

(c) Post-attention conditioning

Ref: Wu & Yang, "MuseMorphose: Full-song and fine-grained piano music style transfer with one Transformer VAE", TASLP 2023

MuseMorphose: VAE-Transformer for Style Transfer

<i>Model</i>	Best ckpt. at step	Recons. NLL		KL divergence	
		<i>Train</i>	<i>Val.</i>	<i>Train</i>	<i>Val.</i>
MIDI-VAE [19]	65K	0.676	0.894	0.697	0.682
Attr-Aware VAE [20]	190K	0.470	0.688	0.710	0.697
MuseMorphose (Ours), AE objective	90K	0.130	0.139	6.927	6.928
MuseMorphose (Ours), VAE objective	75K	0.697	0.765	0.485	0.484
MuseMorphose (Ours), preferred settings	140K	0.457	0.584	0.636	0.636

bold: leads the baselines under the same latent space constraints

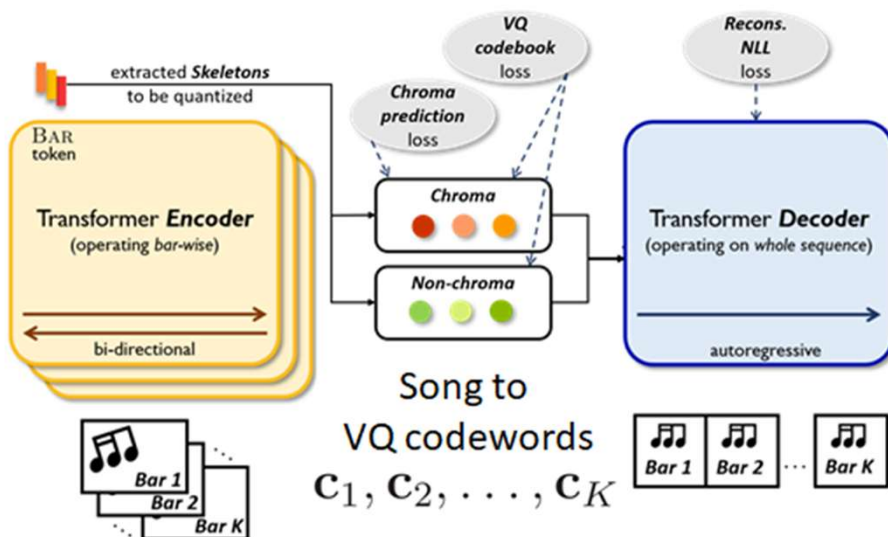
<i>Model</i>	Fidelity		Control		Fluency	Diversity	
	$sim_{chr} \uparrow$	$sim_{grv} \uparrow$	$\rho_{rhytm} \uparrow$	$\rho_{poly} \uparrow$	PPL $\downarrow$	$sim_{chr} \downarrow$	$sim_{grv} \downarrow$
MIDI-VAE [19]	75.6	85.4	.719	.261	8.84	74.8	86.5
Attr-Aware VAE [20]	85.0	76.8	.997	.781	10.7	86.5	84.7
Ours, AE objective	98.5	95.7	.181	.154	6.10	97.9	95.4
Ours, VAE objective	78.6	80.7	.931	.884	7.89	73.2	84.9
Ours, preferred settings	91.2	84.5	.950	.885	7.39	87.1	87.6

Ref: Wu & Yang, “MuseMorphose: Full-song and fine-grained piano music style transfer with one Transformer VAE”, TASLP 2023

VQVAE Transformer for Structured Generation

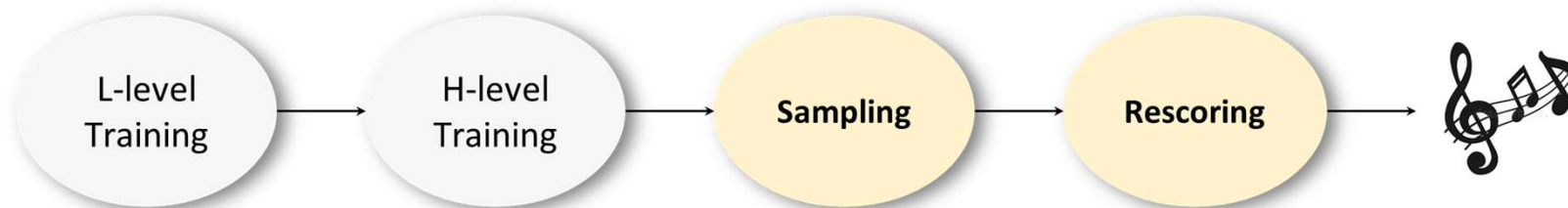
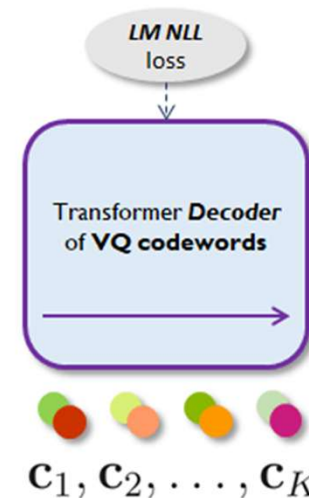
Low-level Model:

Structure Extractor + Conditional Decoder



High-level Model:

Structure Generator



(Slide made by Shih-Lun Wu; unpublished)

VQVAE Transformer Result

- **Conditional generation:** given VQs from songs in test set
- **Better structure?** -- note n-gram repetition stats

		Short (3~6 notes)	Mid (7~15 notes)	Long (16~40 notes)
<i>Unique %</i>	Real data	42.67	60.85	75.49
	VQ model	45.30	64.05	76.95
	Uncond. model	9.32	15.23	27.13
<i>Reappearance interval (in bars)</i>	Real data	16.61	23.68	29.22
	VQ model	16.33	23.74	29.08
	Uncond. model	2.74	3.56	4.70

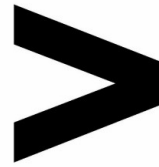
- **Possible plagiarism?** -- len of longest common note string
 - **VQ gen vs. Ref real song:** 6.24 +/- 1.98 notes
 - Random real song pairs: 3.79 +/- 1.49 notes

VQVAE Transformer

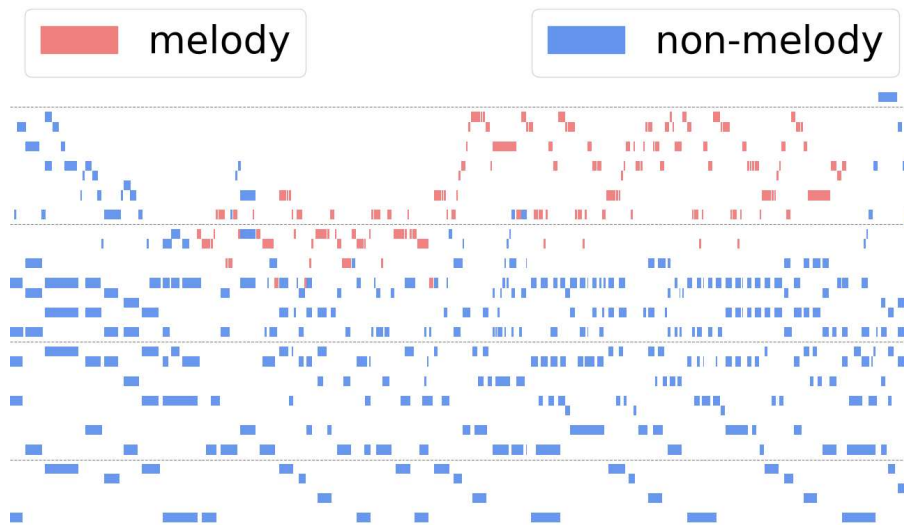
- May allow us to generate a **skeleton/outline** of the music first, before we fill the details
- But
 - Empirical result incomplete (not yet finished)
- Question: The VQVAE approach sounds quite brute-force, is there a **musically-inspired** approach?

Domain Knowledge Inspired Approach

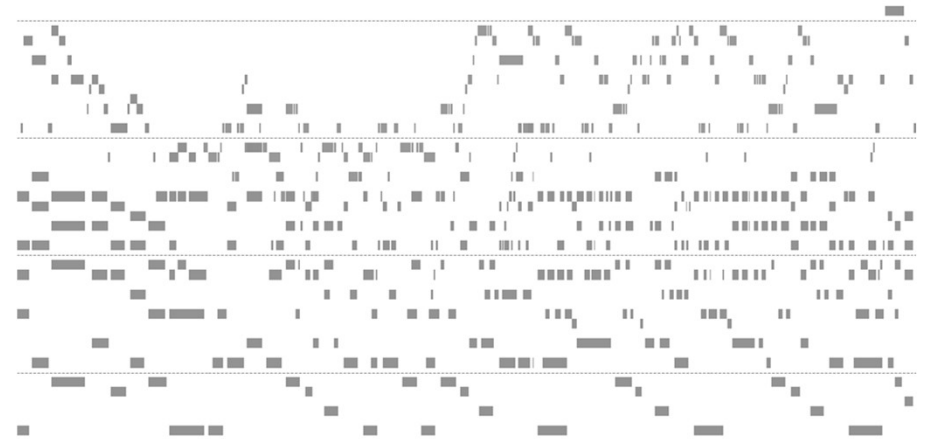
Modular approach



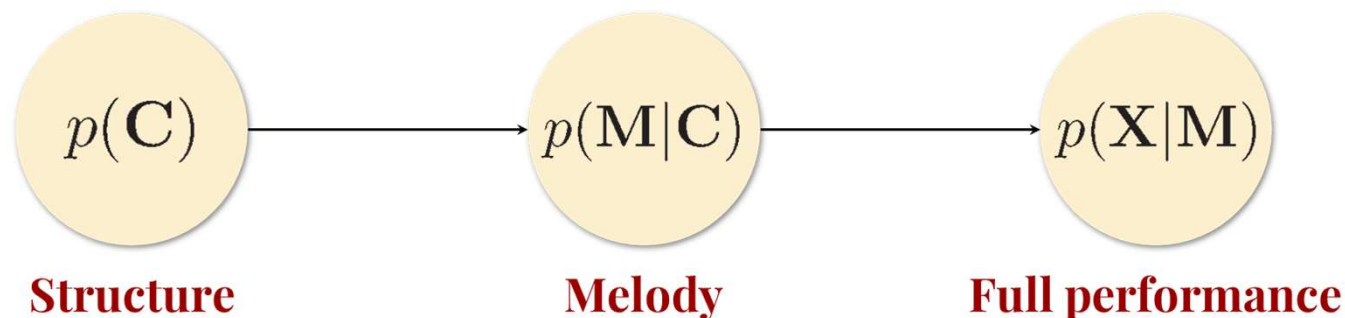
End-to-end approach



Generate melody first,
then the piano accompaniment



Generate Melody First, then the Piano

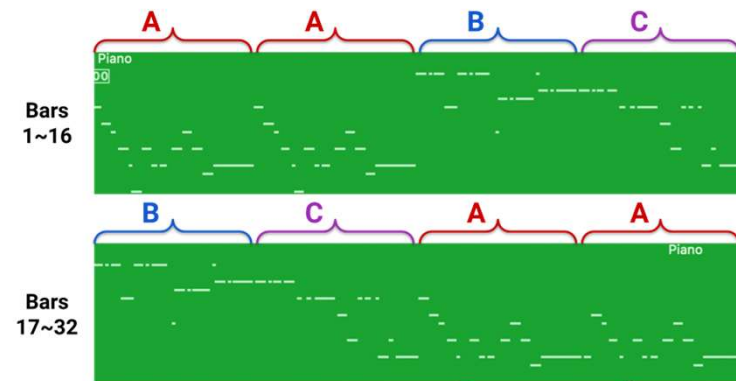


- End-to-end full-song generation is hard; will a 2-stage model make things easier? → Generate **lead sheet** (melody & chords) first, and then the **full piano performance**
- Can we pretrain the melody stage with non-piano data to do better?

Compose & Embellish

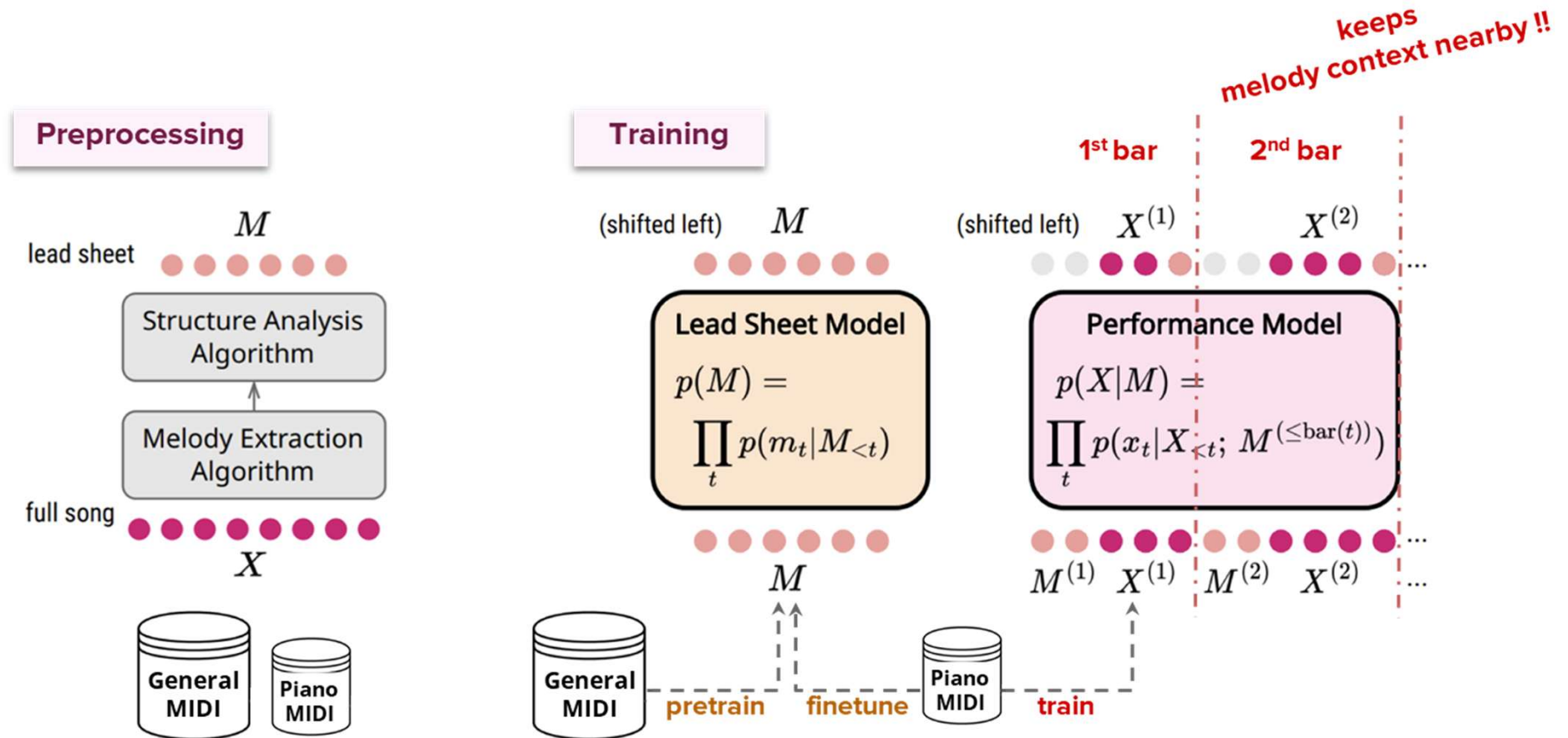


- A two-stage model
 - **Compose** lead sheet & form
 - **Embellish** it to create piano performance



Ref: Wu & Yang, "Compose & Embellish: Well-structured piano performance generation via a two-stage approach", ICASSP 2023

Compose & Embellish

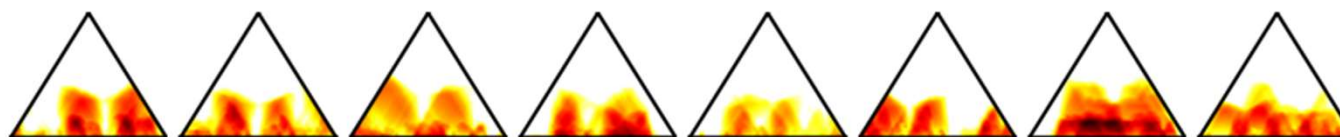


Ref: Wu & Yang, "Compose & Embellish: Well-structured piano performance generation via a two-stage approach", ICASSP 2023

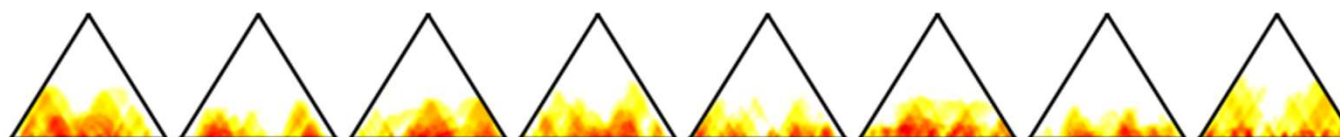
Compose & Embellish: Result

[https://github.com/slSeanWU/Compose and Embellish](https://github.com/slSeanWU/Compose_and_Embellish)

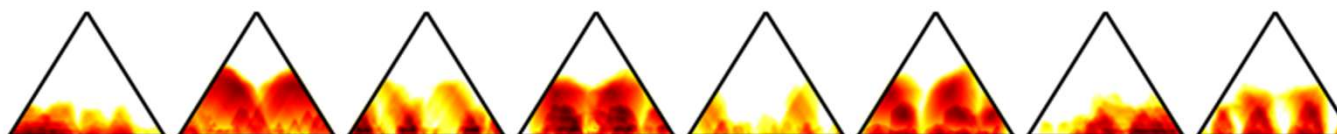
Compose & Embellish (Ours)



CP Transformer (Hsiao et al., '21)



Real Data



Ref: Wu & Yang, "Compose & Embellish: Well-structured piano performance generation via a two-stage approach", ICASSP 2023

Compose & Embellish: Result

- Greatly outperforms CP Transformer in structureness
- Ablations show that both the use of structure tokens (“struct”) and the lead sheet data pre-training help

	Structureness (in %)			Melody-line Diversity		
	$\mathcal{SI}_{\text{short}}$	$\mathcal{SI}_{\text{mid}}$	$\mathcal{SI}_{\text{long}}$	$\mathcal{DN}_{\text{short}}$	$\mathcal{DN}_{\text{mid}}$	$\mathcal{DN}_{\text{long}}$
<i>Naive repeats (1-bar)</i>	83.6 ± 8.6	88.3 ± 5.4	75.7 ± 11	1.2 ± 0.6	1.2 ± 0.6	1.3 ± 0.6
<i>Naive repeats (1-phrase)</i>	75.5 ± 15	86.2 ± 6.8	73.9 ± 11	8.2 ± 4.8	10.0 ± 5.8	10.8 ± 6.5
CP Transformer [4]	32.5 ± 3.3	29.9 ± 4.4	17.9 ± 6.3	82.0 ± 8.9	99.6 ± 0.7	100 ± 0.0
COMPOSE & EMBELLISH	36.8 ± 6.7	35.1 ± 7.7	25.8 ± 12	49.7 ± 19	69.9 ± 20	81.6 ± 17
w/o struct	36.8 ± 6.8	34.2 ± 9.2	23.8 ± 11	48.0 ± 17	68.7 ± 17	82.7 ± 14
w/o pretrain	36.6 ± 7.5	33.1 ± 8.8	19.6 ± 10	53.2 ± 19	74.9 ± 18	87.9 ± 14
w/o struct & pretrain	36.3 ± 6.0	34.1 ± 6.7	23.0 ± 8.4	52.5 ± 18	76.1 ± 16	89.0 ± 11
Real data	43.8 ± 7.1	43.1 ± 8.4	34.8 ± 12	50.0 ± 14	74.1 ± 14	88.3 ± 11

Ref: Wu & Yang, “Compose & Embellish: Well-structured piano performance generation via a two-stage approach”, ICASSP 2023

Compose & Embellish: Result

- 5-point Likert scale on 5 aspects
 - coherence (Ch), correctness (Cr), structureness (S), richness (R), overall (O)

	Ch	Cr	S	R	O
CPT	2.38±0.9	2.49±0.9	2.33±0.9	2.64±0.9	2.33±0.9
C&E (ours)	3.53±0.9	3.11±1.0	3.36±1.2	3.29±1.0	3.18±0.9
Real data	4.42±0.7	4.13±0.8	4.44±0.8	4.24±0.8	4.40±0.7

- Large (>1 point) gain over CP Transformer on coherence & structureness
- Still a long way to go to rival humans

Ref: Wu & Yang, “Compose & Embellish: Well-structured piano performance generation via a two-stage approach”, ICASSP 2023

Recap

- **Theme Transformer**
 - Allow for explicit **conditioning of “theme”** thru customized cross attention design
 - The idea is quite general and applicable to various types of music (melody, single-track piano, multi-track MIDI etc)
- **VAE Transformer (MuseMorphose)**
 - Allow for **time-varying condition**
 - For **style transfer**, and variation generation (as it needs a reference MIDI)
 - The idea can be extended to multi-track MIDI
 - The input can take audio files instead

Recap

- **VQVAE Transformer**

- A brute-force approach for unconditional MIDI generation that generates a **blueprint** first, before filling the details
- Can be extended to multi-track MIDI

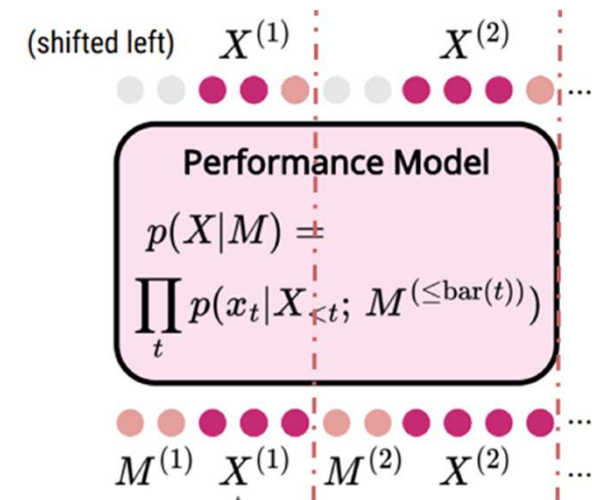
- **Compose-and-embellish**

- A musically-inspired approach for unconditional MIDI generation that generates a ***lead sheet as the blueprint*** first, before filling the details
- More interpretable, interactive; can use unpaired lead sheet data for pretraining
- Can be extended to multi-track MIDI
- Can be combined with Theme Transformer
- The input can take audio files instead

Transformer-based Conditional Generation

	Condition type	Conditioning mechanism
Theme Transformer	token sequence (theme)	cross attention
MuseMorphose	embedding sequence	in-attention
Compose & Embellish	token sequence (lead sheet)	interleaving prompt

- Decoder-only models tends to be easier to train than encoder/decoder models
- Use “bar” for segmentation tends to be useful for symbolic music generation
- Special cares might be needed to avoid the conditions to be ignored



Recommended MIDI Generation Transformers

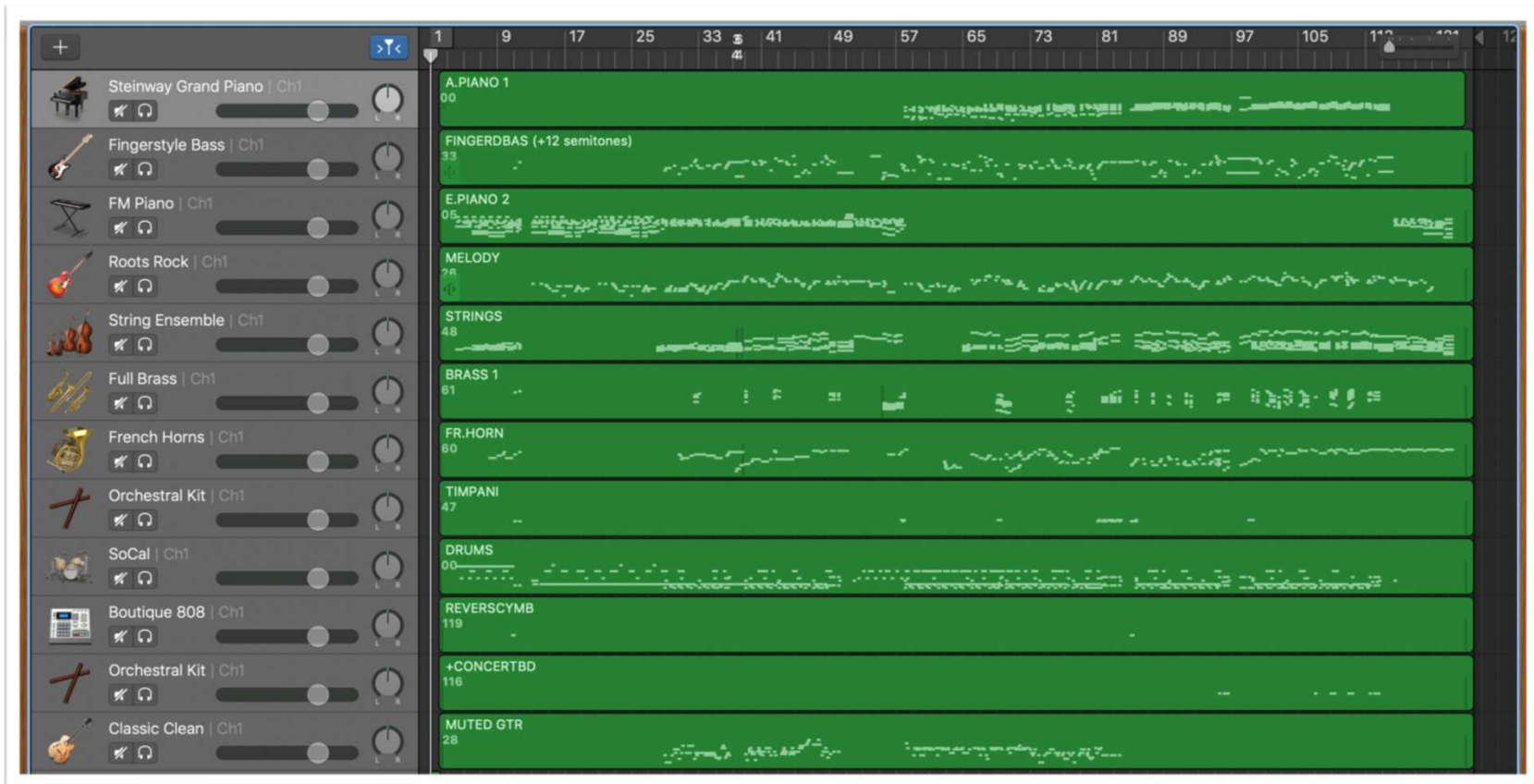
- Piano MIDI performance
 - Pop music transformer
 - Compound word transformer
 - Compose-and-embellish
- Multi-track MIDI score
 - Multitrack music transformer
 - Anticipatory music transformer

Outline

- Text-based approach for symbolic music generation
- Advanced models for MIDI generation
- **Image-based approach for symbolic music generation**
- Symbolic representations for music: Not just MIDI

Two Main Approaches

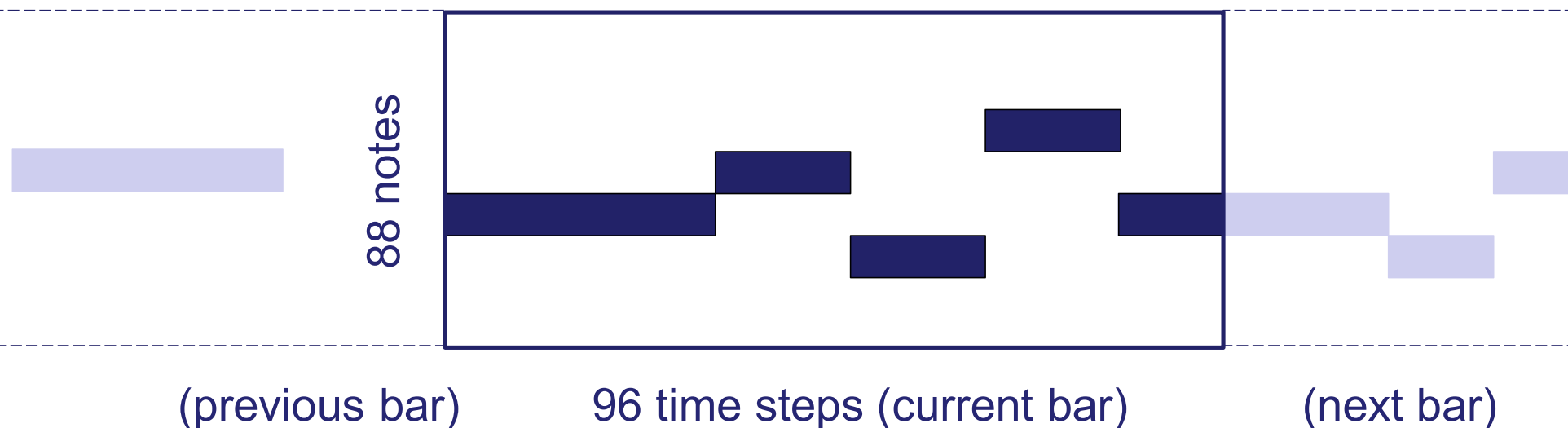
(1) Consider MIDI as **image** using the piano roll representation



(image from the internet)

Piano Roll

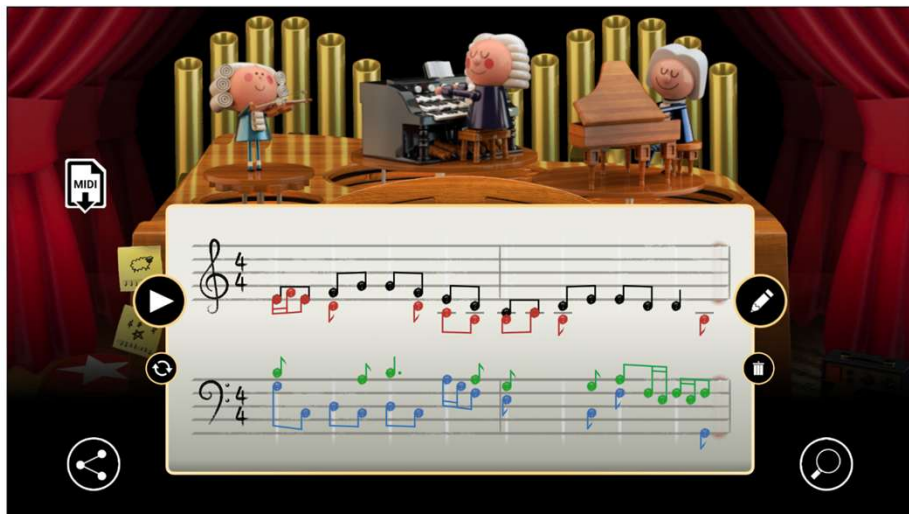
Represent each bar of a single-track MIDI as a fixed-size matrix (thus an image)



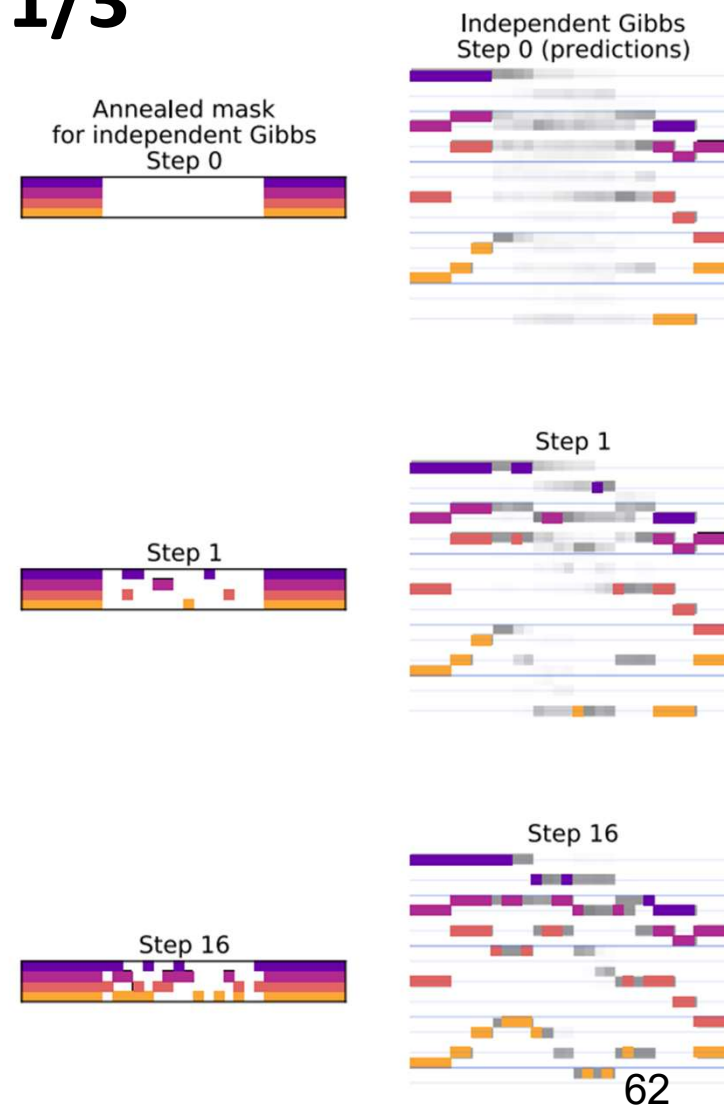
Advantage of Image-based Approach 1/3

- **Non-autoregressive generation**
 - Can do “infilling”: to complete *partial* scores
 - Example: **Bach Doodle**

<https://magenta.tensorflow.org/coconet>



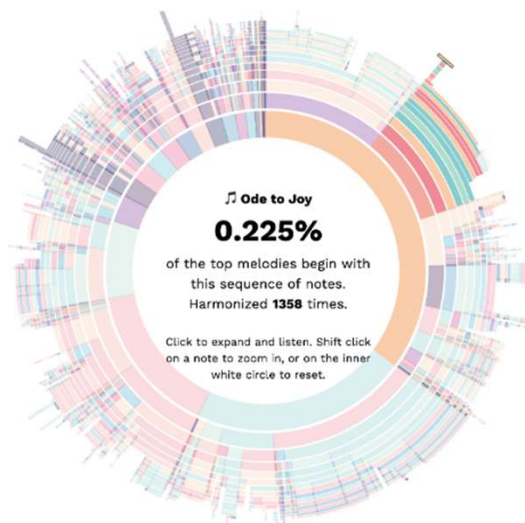
Ref: Huang et al, “Counterpoint by convolution”, ISMIR 2017



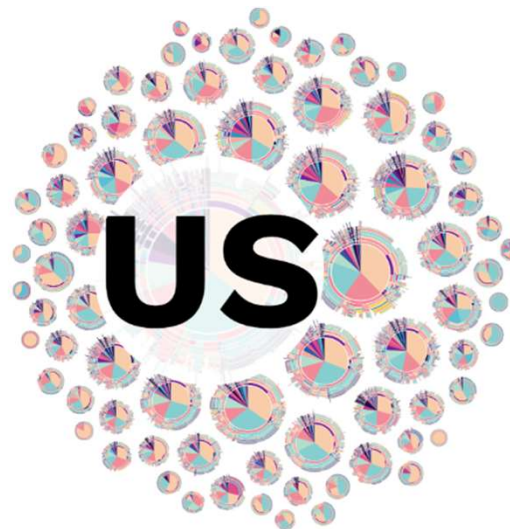
Coconet & Bach Doodle

<https://magenta.tensorflow.org/datasets/bach-doodle>

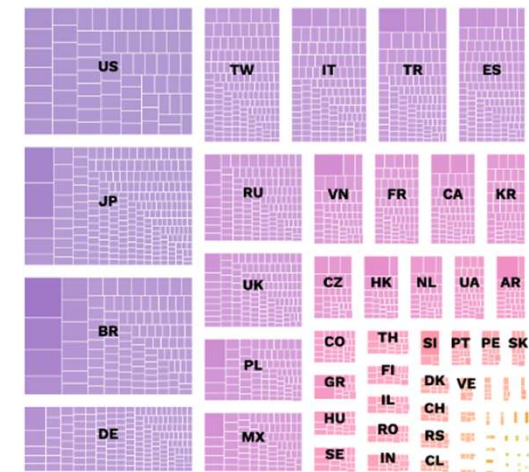
- “In three days, people spent 350 years worth of time playing with the Bach Doodle, and Coconet received more than 55 million queries”



Top overall repeated melodies



Top repeated melodies per country



Unique regional hits

Ref: Huang et al, “The Bach Doodle: Approachable music composition with machine learning at scale”, ISMIR 2019

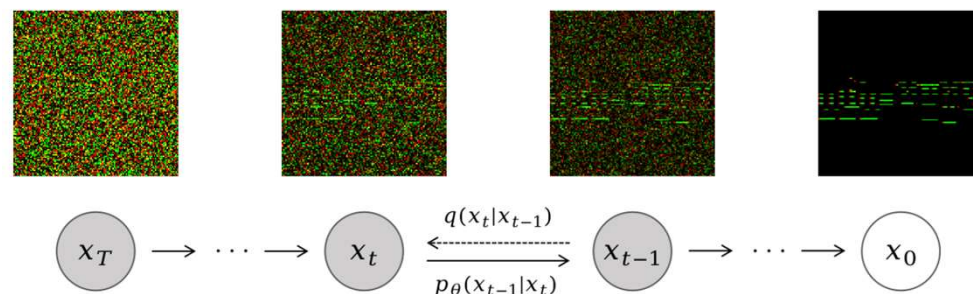
Polyfussion: Diffusion-based Approach

<https://polyffusion.github.io/>

- Work on pixels by treating pianorolls as images

- Support a variety of tasks

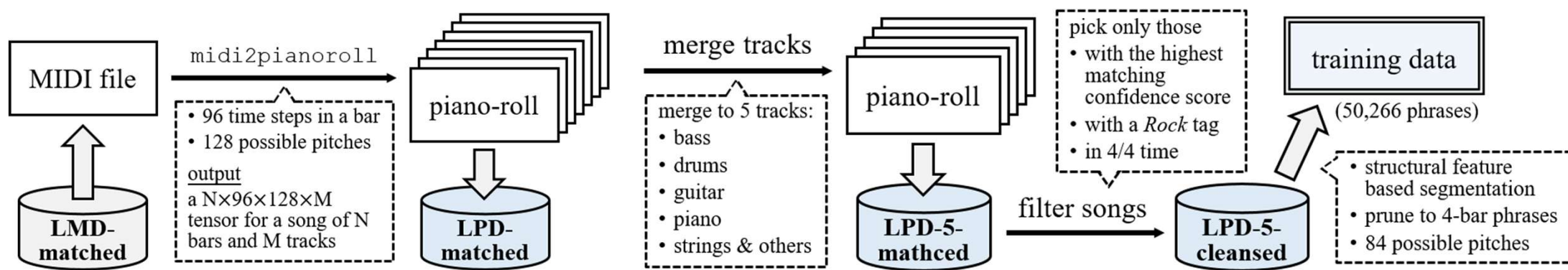
- Unconditional generation
- Melody generation given accompaniment
- Accompaniment generation given melody
- Arbitrary segment inpainting
- Iterative inpainting for long-term generation
- Generation with chord conditioning
- Generation with texture conditioning



Ref: Min et al, "Polyffusion: A Diffusion model for polyphonic score generation with internal and external controls," ISMIR 2023

Advantage of Image-based Approach 2/3

- Straightforward for **multi-track generation**
 - Multi-track piano roll: modeling a fixed-size **tensor**
 - Easy to **synchronize** different instruments/voices
 - Example— LPD dataset: 128K MIDI files (piano-rolls)
<https://salu133445.github.io/musegan/dataset>

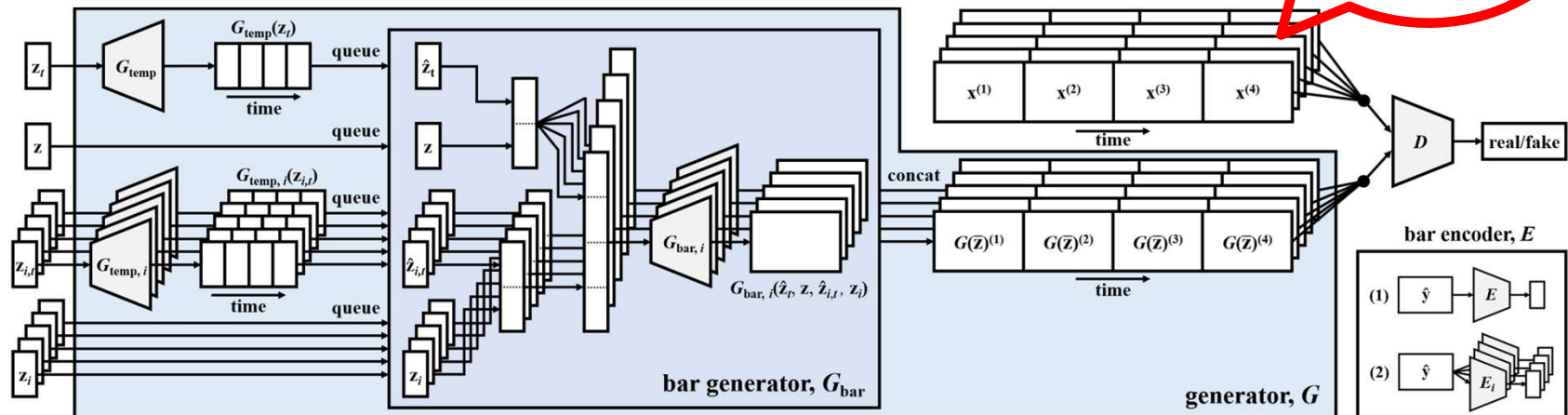


Ref: Dong et al, "MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment", AAAI 2018

MuseGAN

<https://salu133445.github.io/musegan/>

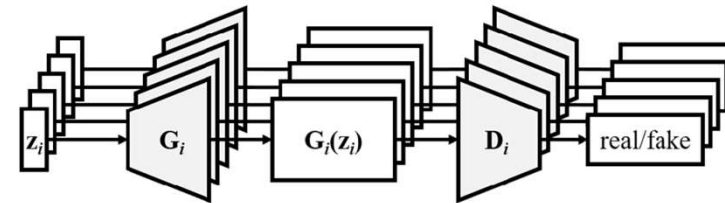
- Convolutional GAN, for **multi-track** MIDI generation
 - 5 tracks, 4 bars
 - Sophisticated design for cross-track, cross-bar dependency



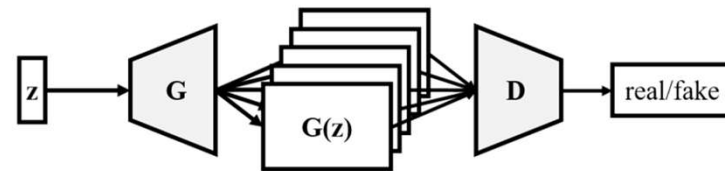
Ref: Dong et al, "MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment", AAAI 2018

MuseGAN: Intra- & Inter-tracks

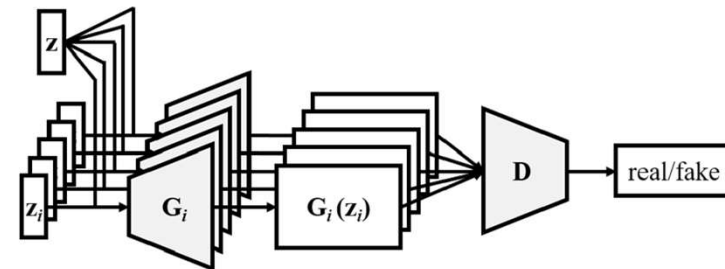
- Multi-track
 - Piano, guitar, bass, strings, drums
- **Hybrid model**
 - One “**shared**” (inter) z
 - Five “**private**” (intra) z_i
 - **Five** generators
 - **One** discriminator



(a) the jamming model



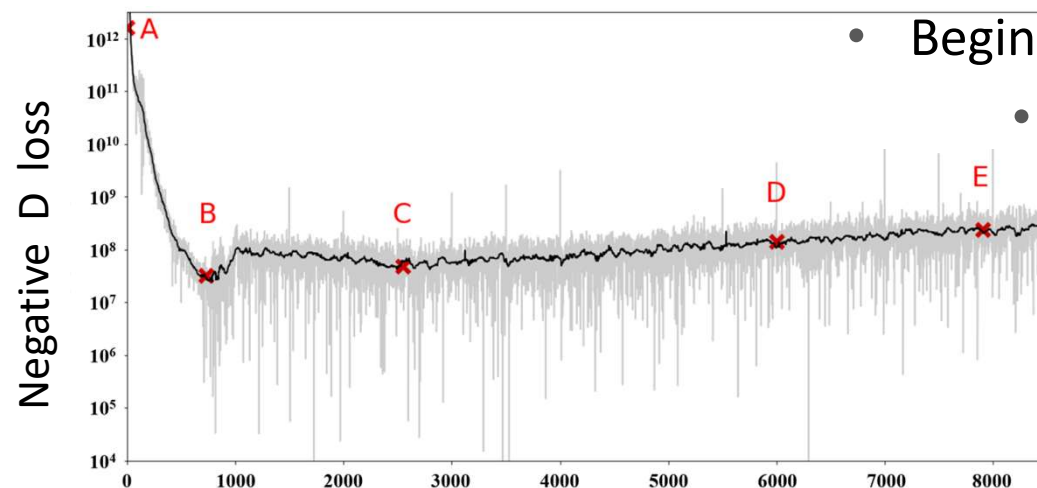
(b) the composer model



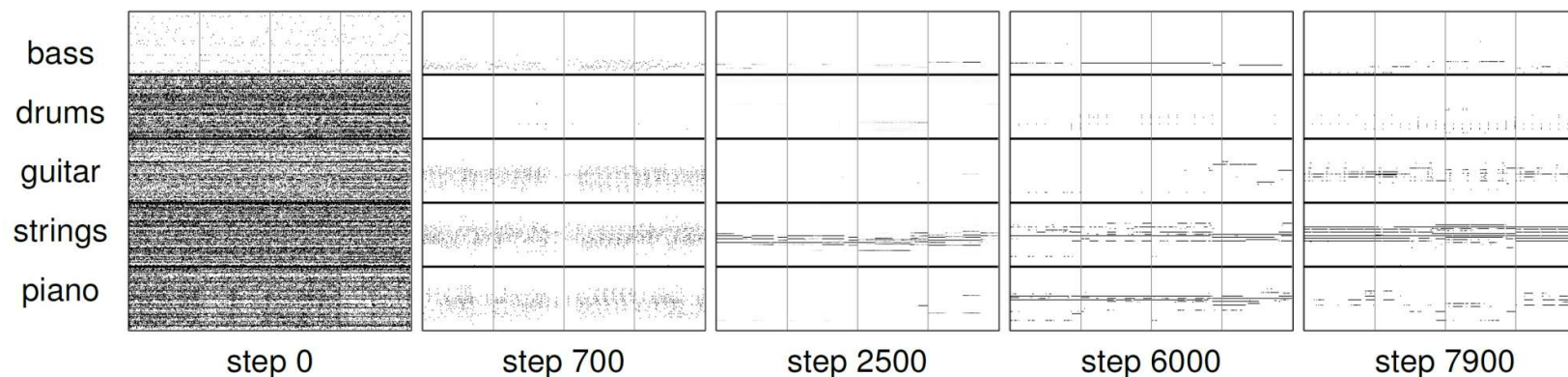
(c) the hybrid model

Ref: Dong et al, “MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment”, AAAI 2018

MuseGAN: WGAN-gp



- Beginning – 500:1 D/G updates
- Later – 5:1 D/G updates
- Training time: <24 hours



Ref: Dong et al, "MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment", AAAI 2018

MuseGAN: G & D

Input: $\mathbf{z} \in \mathbb{R}^{128}$					
reshaped to $(1, 1) \times 128$ channels					
transconv	1024	2×1	$(2, 1)$	BN	ReLU
transconv	256	2×1	$(2, 1)$	BN	ReLU
transconv	256	2×1	$(2, 1)$	BN	ReLU
transconv	256	2×1	$(2, 1)$	BN	ReLU
transconv	128	3×1	$(3, 1)$	BN	ReLU
transconv	64	1×7	$(1, 7)$	BN	ReLU
transconv	K_{bar}	1×12	$(1, 12)$	BN	tanh
Output: $G_{\text{bar}}(\mathbf{z}) \in \mathbb{R}^{96 \times 84 \times K_{\text{bar}}}$ (K_{bar} -track piano-roll)					

(b) the bar generator G_{bar}

- **Grow time steps first**
 - $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 96$
- **Then notes (freq)**
 - octave (7)
 - then, pitch (84)

Ref: Dong et al, "MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment", AAAI 2018

Input: $\tilde{\mathbf{x}} \in \mathbb{R}^{4 \times 96 \times 84 \times 5}$ (real/fake piano-rolls of 5 tracks)				
reshaped to $(4, 96, 84) \times 5$ channels				
conv	128	$2 \times 1 \times 1$	$(1, 1, 1)$	LReLU
conv	128	$3 \times 1 \times 1$	$(1, 1, 1)$	LReLU
conv	128	$1 \times 1 \times 12$	$(1, 1, 12)$	LReLU
conv	128	$1 \times 1 \times 7$	$(1, 1, 7)$	LReLU
conv	128	$1 \times 2 \times 1$	$(1, 2, 1)$	LReLU
conv	128	$1 \times 2 \times 1$	$(1, 2, 1)$	LReLU
conv	256	$1 \times 4 \times 1$	$(1, 2, 1)$	LReLU
conv	512	$1 \times 3 \times 1$	$(1, 2, 1)$	LReLU
fully-connected	1024			LReLU
fully-connected	1			
Output: $D(\tilde{\mathbf{x}}) \in \mathbb{R}$				

(c) the discriminator D

Advantage of Image-based Approach 3/3

- **Fixed size**
 - Generating symbolic music becomes similar to generating images
 - Long music → high-resolution image

Whole-song Hierarchical Generation of Symbolic Music

<https://wholesonggen.github.io/>

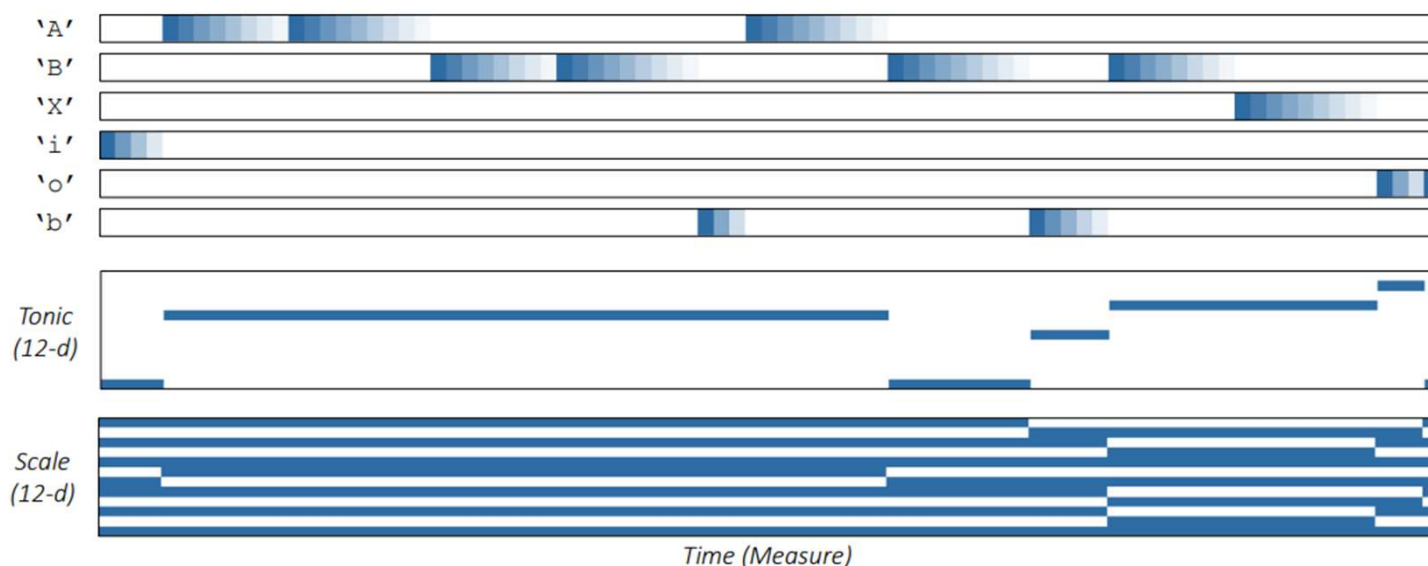
Table 1: Definition of the four-level hierarchical music language. We use m for measure, b for beat, s for step, to represent the temporal resolution. M denotes the number of measures in a piece, γ denote the number of beats in a measure, and δ denotes the number of steps in a beat.

Languages (res.)	Specification	Data Representation	Structural Focus
Form (m)	Key changes Phrases	$\mathbf{X}^1 \in \mathbb{R}^{8 \times M \times 12}$	Music form
Reduced Lead Sheet (b)	Melody reduction Simplified chord	$\mathbf{X}^2 \in \mathbb{R}^{2 \times \gamma M \times 128}$	Phrase similarity, phrase development & cadence
Lead Sheet (s)	Lead melody Chord	$\mathbf{X}^3 \in \mathbb{R}^{2 \times \delta \gamma M \times 128}$	Melodic pattern, similarity & coherence
Accompaniment (s)	Accompaniment	$\mathbf{X}^4 \in \mathbb{R}^{2 \times \delta \gamma M \times 128}$	Acc. pattern, similarity & coherence, Mel-acc relations

Whole-song Hierarchical Generation of Symbolic Music

- Stage 1:
Form

(i8) (A8B16A8B16) (b6) (B14) (o2)
 (i12) (A4A4B12) (b4b4) (A4A4B12) (b4b4 (B16) o4o1)
 (i4) (A4A4B4X5) (b4) (A4b5B4X4X5) (o2)
 (i4) (A8B9A8B9X18) (o2o1)
 (i8) (A4A4B4B4) (b7) (A4A4B4B4B4B4) (o4)
 (i8) (A4A4B4B4) (b7) (A4A4B4B4B4B4) (o4)
 (i8) (A8B8X8X8) (b4b4) (A8B8X8X8X4) (o6o1)
 (i4) (A4A4B9) (b3) (A4B10B9X1) (o2)
 (i4) (A4A4B9) (b3) (A4B10B9X1) (o2)
 (i12) (A16B16) (b4b4b4) (A16B16B16) (o10o1)



Ref: Wang et al, "Whole-song hierarchical generation of symbolic music using cascaded diffusion models," ICLR 2024

Whole-song Hierarchical Generation of Symbolic Music

- Stage 2:
**Reduced
lead sheet**

The musical score consists of eight staves, each with a melody line and a sequence of chords written above it. The chords are as follows:

- a) $E\flat\Delta 7/B\flat$, $E\flat\text{ sus}2$, $A\flat\Delta 7$, $A\flat$, $E\flat\Delta 7$, $E\flat\Delta 7/D$, $A\flat\Delta 7$, $Fm7$, $B\flat 7$
- b) $E\flat$, $E\flat$, $E\flat$, Fm , Fm , Gm , Gm , Cm , Fm , $B\flat\text{ sus}2$
- c) $E\flat\text{ sus}2$, $E\flat$, $A\flat 6$, $B\flat$, $E\flat$, $Cm7$, $B\flat$, Gm , $B\flat$
- d) $E\flat$, Cm , Cm , $B\flat$, $E\flat$, $E\flat$, Cm , Cm , $Cm7/E\flat$, $B\flat$
- e) $E\flat\text{ sus}4$, Gm , $A\flat$, $B\flat$, $E\flat$, $D\flat$, Gm , Cm , Fm , $B\flat$
- f) $E\flat$, $Cm7$, $E\flat$, Cm , $E\flat$, Cm , Fm , $B\flat$
- g*) $E\flat$, $E\flat$, $E\flat$, $E\flat$, $E\flat$, $E\flat$, $E\flat$, $B\flat$
- h*) $E\flat$, $E\flat\Delta 7$, $E\flat$, $E\flat/B\flat$, $E\flat$, $E\flat$, $E\flat$, $E\flat$

Ref: Wang et al, "Whole-song hierarchical generation of symbolic music using cascaded diffusion models," ICLR 2024

Whole-song Hierarchical Generation of Symbolic Music

- Stage 3:
Lead sheet

a) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

b) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

c) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

d) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

e) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

f) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

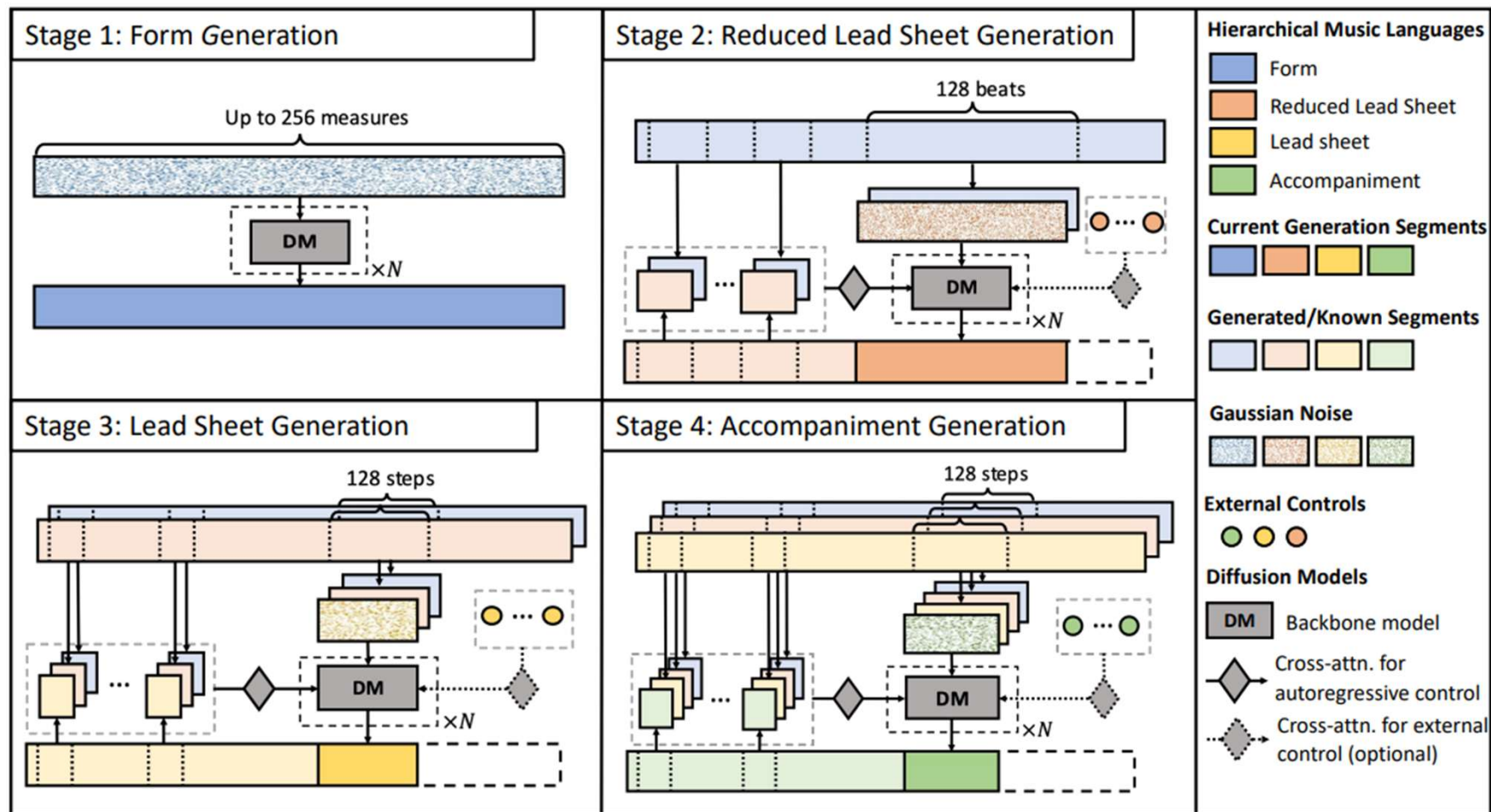
g) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

h*) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

i*) Ebsus2 Gm Cm Ab Eb Ab Bb Ab Bb Gm Cm Fm Gm Ab Bb

Ref: Wang et al, "Whole-song hierarchical generation of symbolic music using cascaded diffusion models," ICLR 2024

Whole-song Hierarchical Generation of Symbolic Music



Ref: Wang et al, "Whole-song hierarchical generation of symbolic music using cascaded diffusion models," ICLR 2024

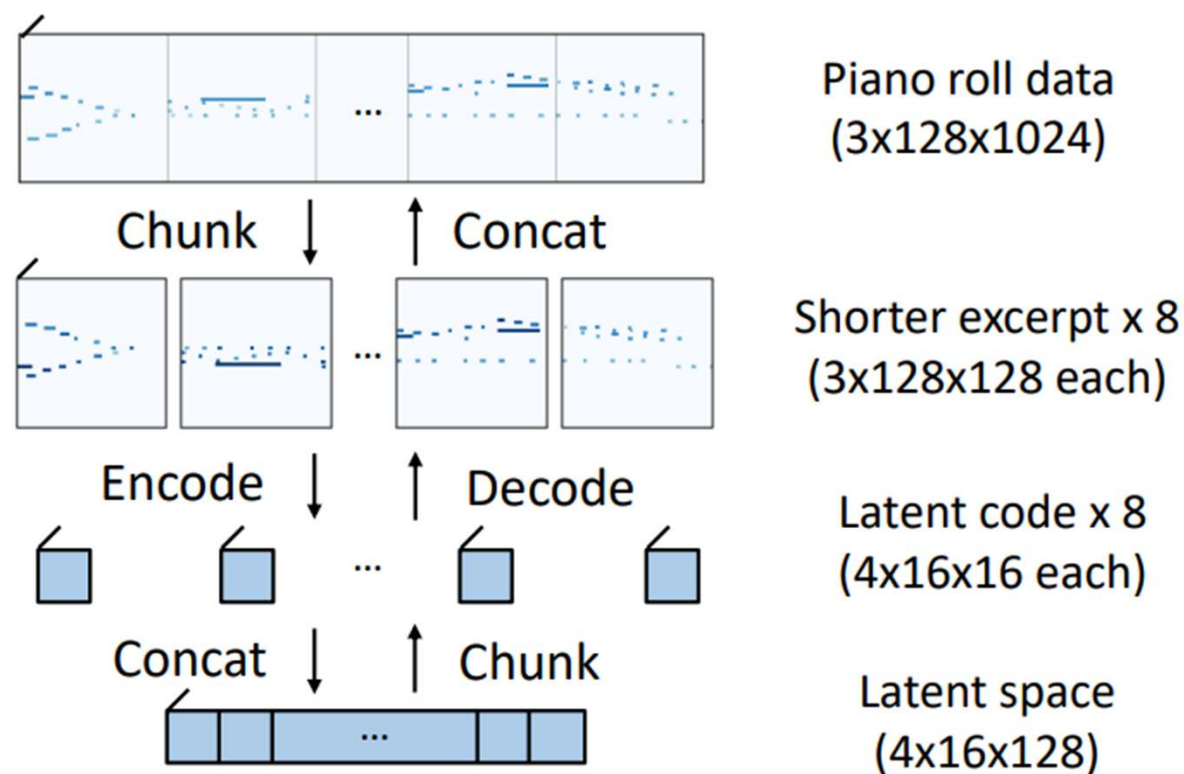
An Issue to Be Considered: Representing Onset

- A long note vs. two repeating notes?
- How to represent “**onset**”?



Representing Onsets using an Additional Channel

- Represent symbolic music as a **3-channel** tensor
 - **Piano roll**
 - takes value from 0-127 indicating note velocity
 - **Onset roll**
 - binary values that denote the presence of note onsets
 - **Pedal roll**
 - representing the sustain pedal control for each timeframe

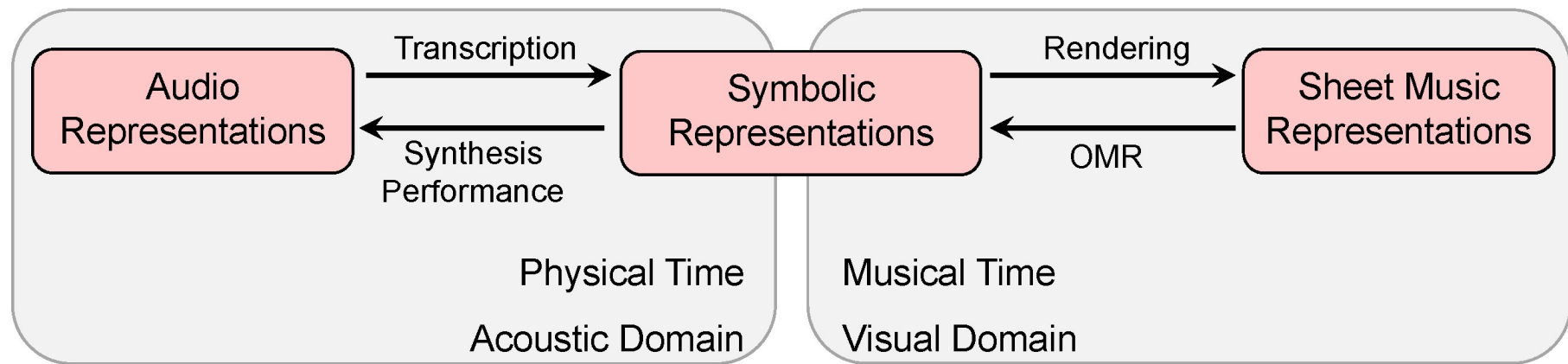


Outline

- Text-based approach for symbolic music generation
- Advanced models for MIDI generation
- Image-based approach for symbolic music generation
- **Symbolic representations for music: Beyond MIDI**

Data Representation of Music

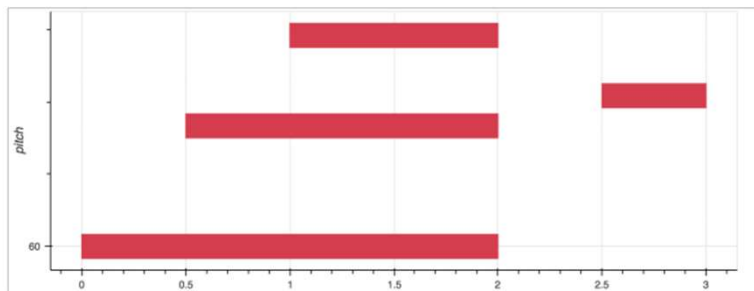
(Figure source: <https://www.mdpi.com/2624-6120/2/2/18>)



- **Musical audio** (waveforms, spectrograms)
 - what we “hear”
 - involve sounds
- **Symbolic representations** (e.g., **MIDI**, **MusicXML**)
 - the way music can be represented in a Digital Audio Workstation (DAW)
 - still relevant today as it’s *interpretable* and *editable*
- **Sheet music** (visual representations of a musical score)
 - what musicians “compose”

MIDI Representation

- A **text-like** representation of *.MIDI
- **Widely** used by deep generative neural networks, especially Transformers
 - By viewing the MIDI messages as “**tokens**”
 - *Music Transformer* (2019) : <https://magenta.tensorflow.org/music-transformer>
 - *MuseNet* (2019): <https://openai.com/research/musenet>
 - *Pop Music Transformer* (2020): <https://github.com/YatingMusic/remi>
 - *MuseCoco* (2023): <https://ai-music.github.io/musecoco/>

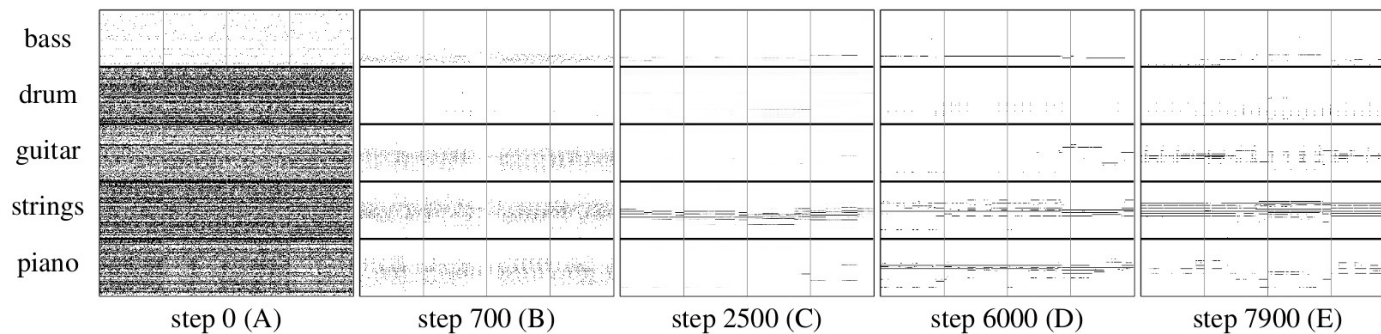


```
SET_VELOCITY<80>, NOTE_ON<60>  
TIME_SHIFT<500>, NOTE_ON<64>  
TIME_SHIFT<500>, NOTE_ON<67>  
TIME_SHIFT<1000>, NOTE_OFF<60>, NOTE_OFF<64>,  
NOTE_OFF<67>  
TIME_SHIFT<500>, SET_VELOCITY<100>, NOTE_ON<65>  
TIME_SHIFT<500>, NOTE_OFF<65>
```

Piano Roll Representation

- A visual, **image-like** representation of *.MIDI
- Multi-track MIDI → multiple piano rolls (i.e., a tensor)
- **Widely** used by deep generative neural networks
 - *MidiNet* (2017) [GAN-based]
 - *MuseGAN* (2018) [GAN-based]: <https://salu133445.github.io/musegan/results>
 - *Polyffusion* (2023) [diffusion-based]: <https://polyffusion.github.io/>

MuseGAN



DiffRoll



MusicXML

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C1/C1S2_MusicXML.html

- A **text-like** representation of **sheet music**

- To represent a <note>

- <pitch>: <step>, <alter> <octave>
 - <duration>
 - <type>

```
<note>
  <pitch>
    <step>E</step>
    <alter>-1</alter>
    <octave>4</octave>
  </pitch>
  <duration>2</duration>
  <type>half</type>
</note>
```



- Can be **tokenized**

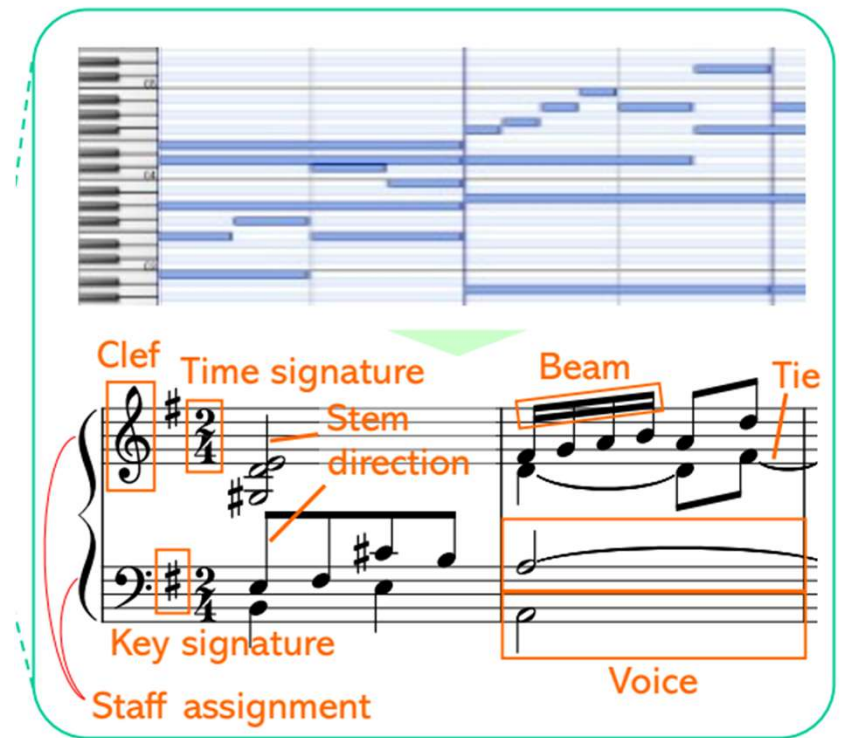
- Musical (not physical) onset times are specified

- Avoid information loss of the piano roll representation

- Example 1, the notes **D#4** and **Eb4** are distinguishable in MusicXML but not in MIDI

MusicXML

- A ***text-like*** representation of **sheet music**
- Readily tokenized
- Musical (not physical) onset times
- Avoid information loss of the piano roll representation
 - Example 1, D $\sharp$ 4 and E $\flat$ 4 (enharmonic equivalents; 同音異名)
 - <https://www.youtube.com/watch?v=Sm-2e2DsdhE>
 - Example 2: can describe **voices**



Ref: Suzuki, "Score Transformer: Generating musical score from note-level representation," MMAsia 2021

Voices

- SATB
 - S: soprano
 - A: alto
 - T: tenor
 - B: bass

Aus tiefer Not First Section

J. S. Bach



Voice type

Female

Soprano
Mezzo-soprano
Contralto

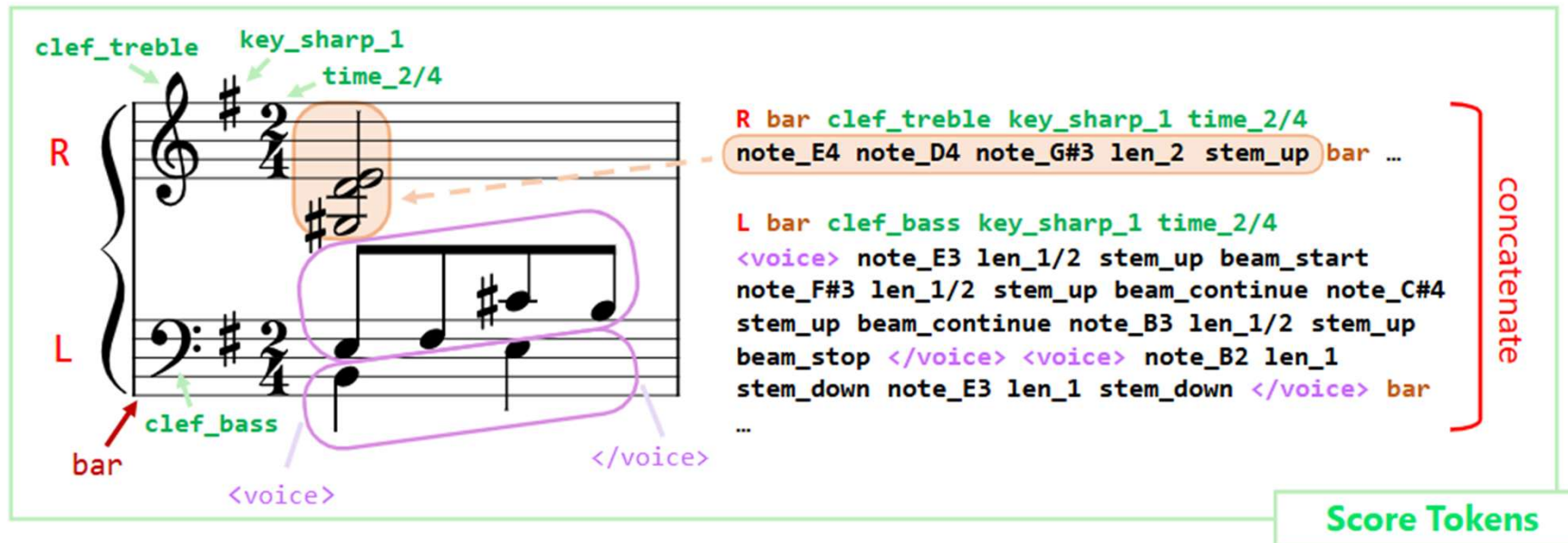
Male

Countertenor
Tenor
Baritone
Bass

<https://www.youtube.com/watch?v=WD0jd1QcMQE>

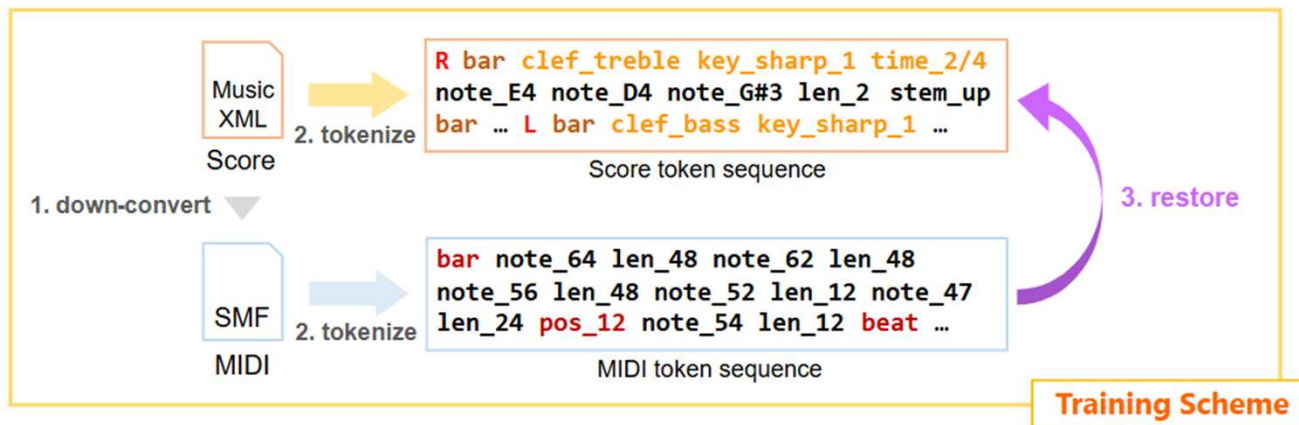
MusicXML

- A ***text-like*** representation of **sheet music**
- *Relatively less* used by deep generative neural networks
 - *Score Transformer* (2021)



Score Transformer

<https://score-transformer.github.io/>



- Train to **translate MIDI into musical scores**
- Expression symbols (e.g., articulations, dynamics, and ornaments) and repeat symbols (can be but) are not included

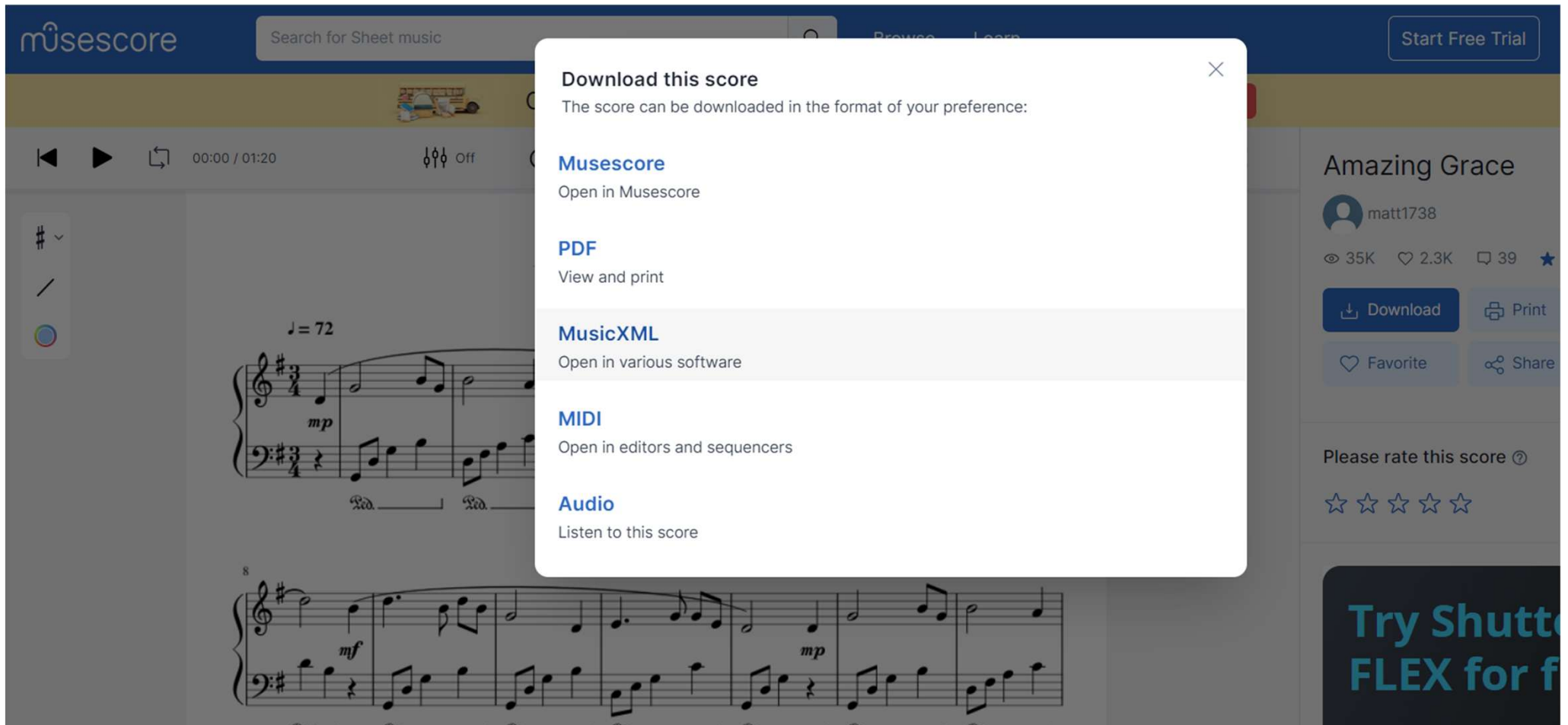
Symbol	Example	Variations
Staff	<i>R</i>	<i>R/L</i>
Barline	<i>bar</i>	<i>bar</i>
Clef	<i>clef_treble</i>	<i>clef_{bass/treble}</i>
Key Signature	<i>key_flat_2</i>	<i>key_{sharp/flat/natural}_{1, 2, ..., 6}</i>
Time Signature	<i>time_4/4</i>	<i>time_{2/4, 3/4, 4/4, etc.}</i>
Voice	<i><voice></i>	<i><voice>, </voice></i>
Rest	<i>rest</i>	<i>rest</i>
Pitch	<i>note_C4</i>	<i>note_{A, B, ..., G}_{##/b/bb/(none)}{0, 1, ..., 8}</i>
Duration	<i>len_1/2</i>	<i>len_{1/24, 1/16, ..., 4}</i>
Stem Direction	<i>stem_up</i>	<i>stem_{up/down}</i>
Beams	<i>beam_stop</i>	<i>beam_{start/stop/continue/partial-left/partial-right}_...</i>
Tie	<i>tie_start</i>	<i>tie_{start/continue/stop}</i>

MIDI-to-Score Conversion

<https://score-transformer.github.io/>



Software: MuseScore

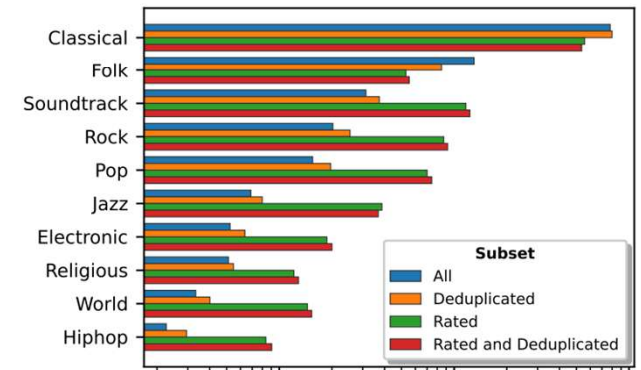


Dataset: PDMX

<https://pnlong.github.io/PDMX.demo/>

Dataset	Format	Hours	Size	Multitrack	Dataset License Composition
Lakh MIDI (LMD) [18]	MIDI	>9,000	174,533	✓	CC-BY 4.0 [†]
SymphonyNet [13]	MIDI	>3,200	46,359	✓	
MAESTRO [19]	MIDI	201.21	1,282		
Wikifonia Lead Sheet Dataset [14]	MusicXML	198.40	6,405		CC BY-NC-SA 4.0
POP909 [15]	MIDI	60.00	909		
EMOPIA [20]	MIDI	11.0	1,078		
MMD [16]	MIDI	-	1,524,557		
MetaMidi [17]	MIDI	-	612,088	✓	
PDMX	MusicXML	6,250	254,077	✓	CC-0 / Public Domain

Subset	Hours / Size	PCE	SC (%)	GC (%)
A	6,250 / 254K	2.69 ± 0.00	97.12 ± 0.01	93.75 ± 0.01
D	3,756 / 102K	2.77 ± 0.00	95.15 ± 0.02	93.79 ± 0.01
R	1,001 / 14K	2.91 ± 0.00	91.59 ± 0.07	92.59 ± 0.05
R∩D	941 / 13K	2.90 ± 0.00	91.70 ± 0.07	92.65 ± 0.05
Fine-Tuning	595 / 6K	2.95 ± 0.00	90.54 ± 0.11	92.39 ± 0.07



Long et al, “PDMX: A large-scale public domain MusicXML dataset for symbolic music processing,” ICASSP 2025

[Recap] Music Analysis Demo 1: Music Transcription

<https://songscription.ai/>



- songscription.ai
- AnthemScore
- Melody Scanner
- Klangio

Flight of the Bumblebee
Nikolai Rimsky-Korsakov

Up Down Scale
Exercise

Invention No. 4 in D Minor
Johann Sebastian Bach

Flight of the Bumblebee
Nikolai Rimsky-Korsakov

Original Audio

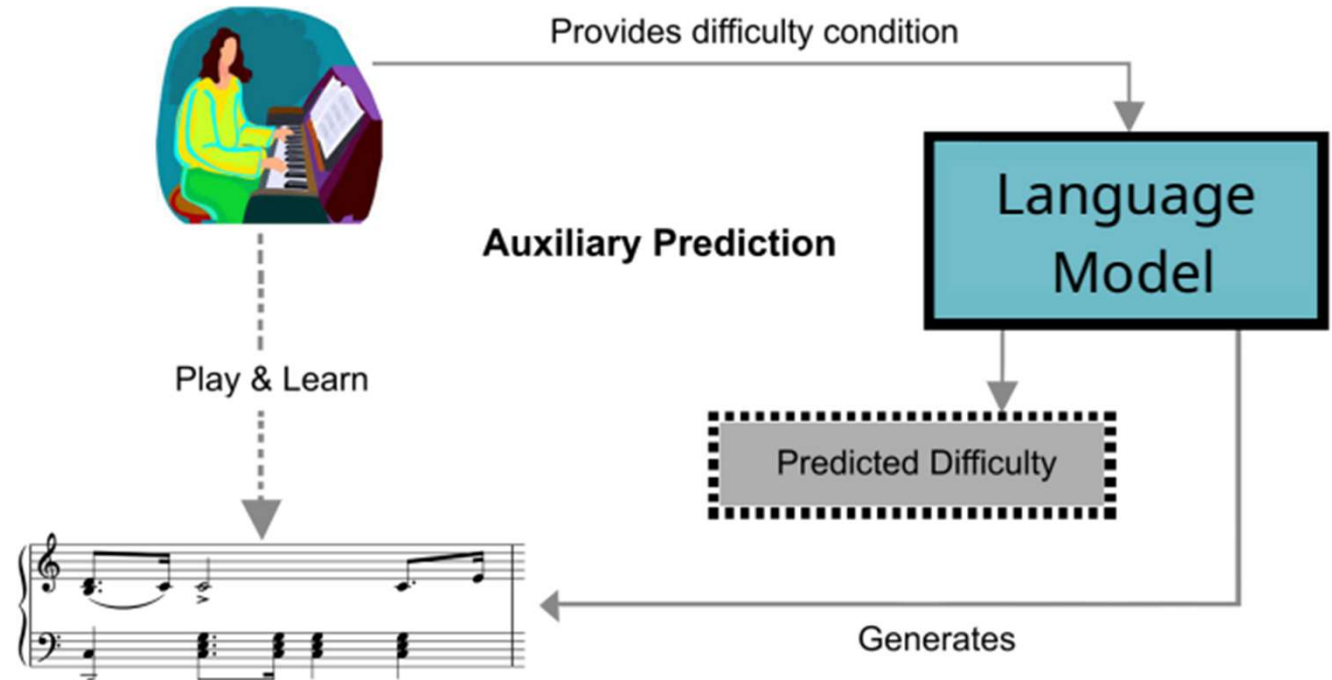
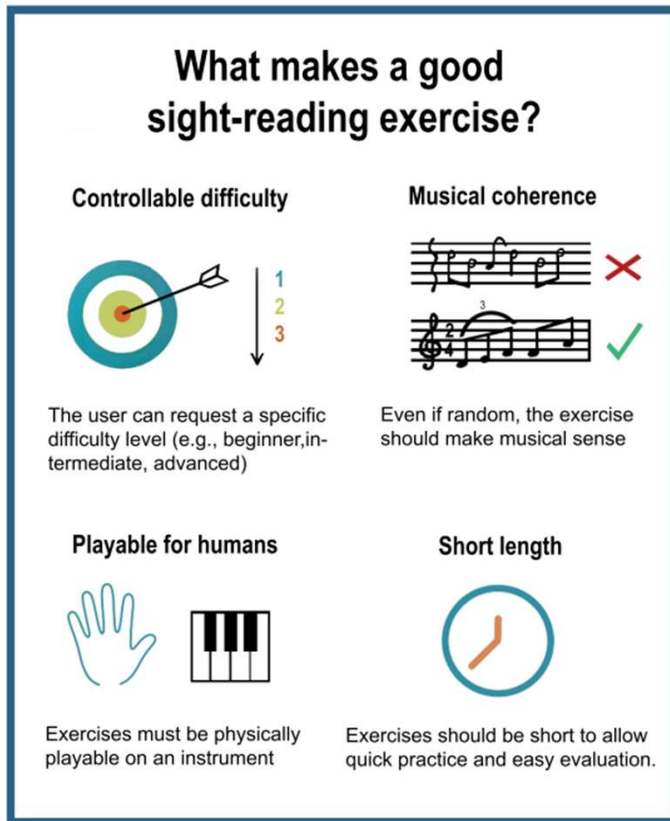
0:00 / 0:22

Play Generated Sheet Music

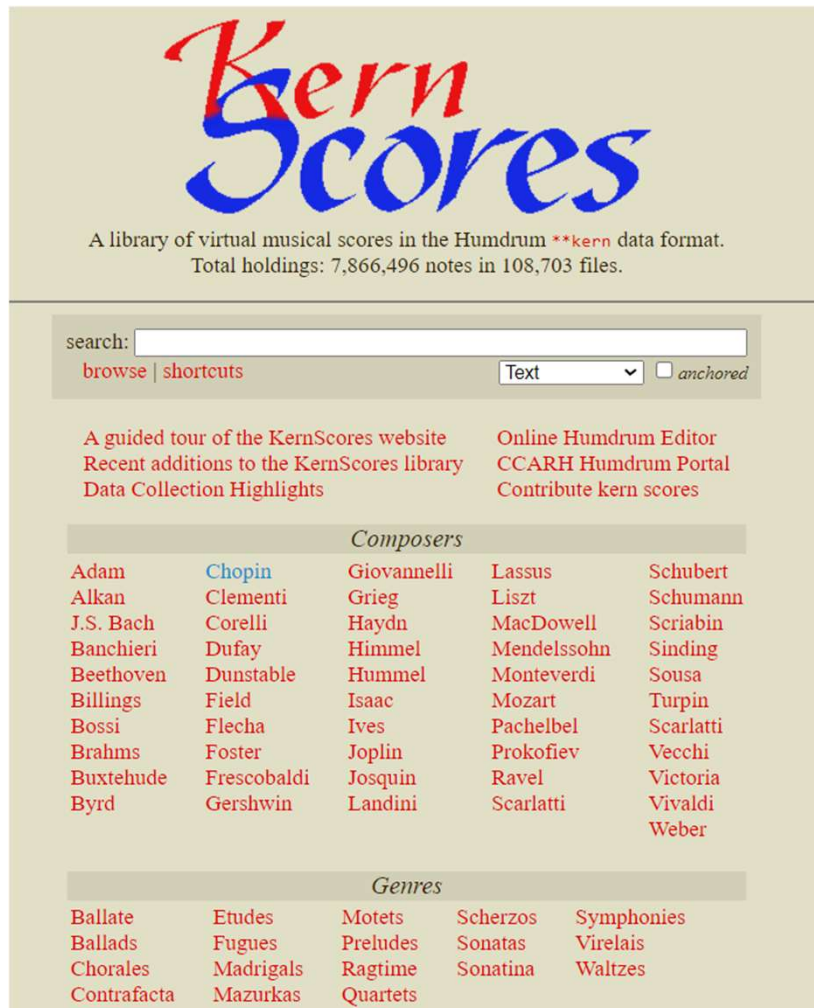
Flight-of-the-bumblebee-piano

The image shows a musical score for 'Flight of the Bumblebee' in piano. It features three systems of music, each with a treble and bass staff. The first system shows the beginning of the piece with a treble staff containing a series of eighth and sixteenth notes, and a bass staff with a whole note chord. The second system continues the melody in the treble staff. The third system shows a more complex texture with both staves containing active music. A vertical scrollbar is on the right, and a horizontal scrollbar is at the bottom.

MusicXML Generation



Software: KernScores



Kern Scores

A library of virtual musical scores in the Humdrum ****kern** data format.
Total holdings: 7,866,496 notes in 108,703 files.

search: browse | shortcuts Text ☐ anchored

A guided tour of the KernScores website
Recent additions to the KernScores library
Data Collection Highlights

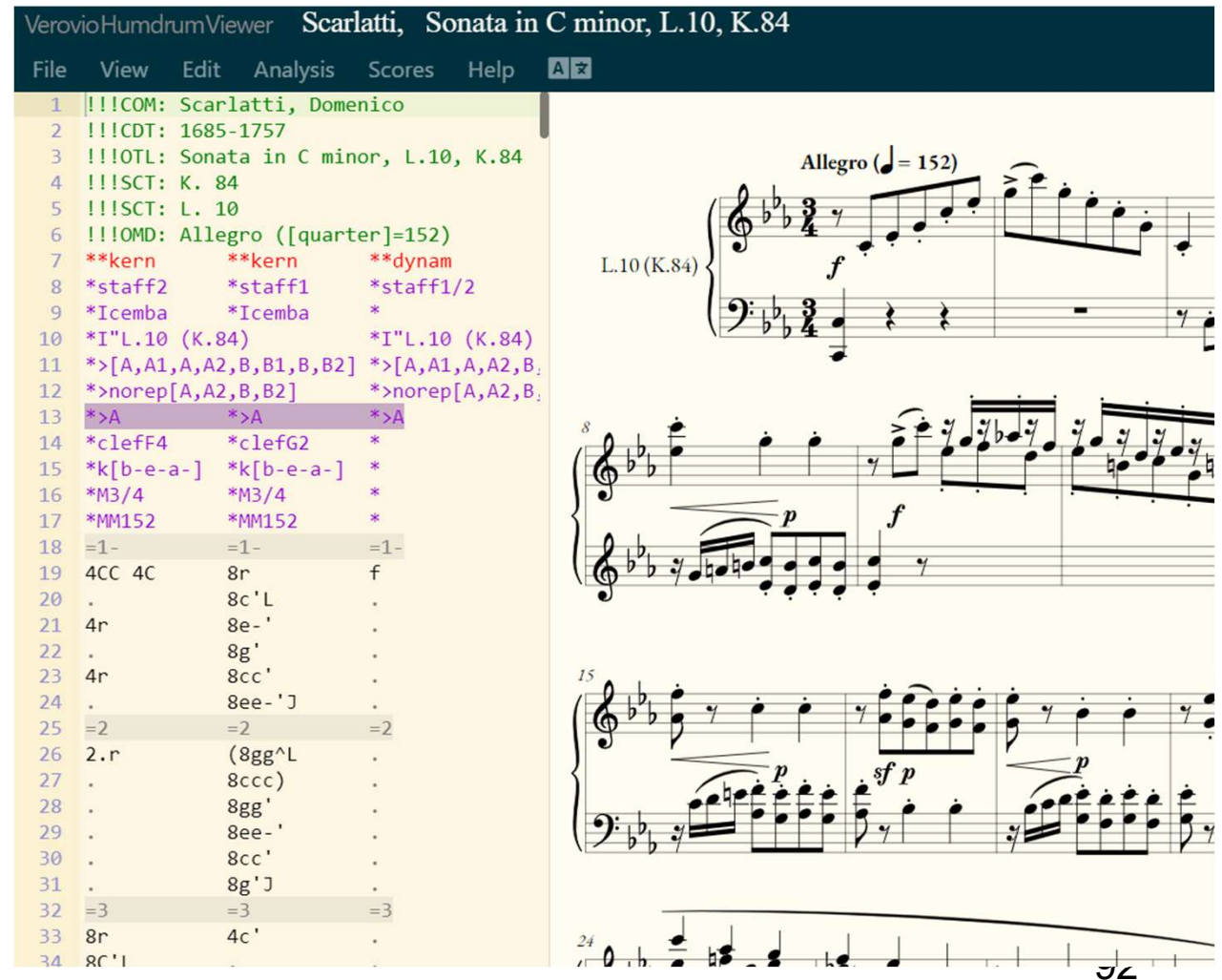
Online Humdrum Editor
CCARH Humdrum Portal
Contribute kern scores

Composers

Adam	Chopin	Giovannelli	Lassus	Schubert
Alkan	Clementi	Grieg	Liszt	Schumann
J.S. Bach	Corelli	Haydn	MacDowell	Scriabin
Banchieri	Dufay	Himmel	Mendelssohn	Sinding
Beethoven	Dunstable	Hummel	Monteverdi	Sousa
Billings	Field	Isaac	Mozart	Turpin
Bossi	Flecha	Ives	Pachelbel	Scarlatti
Brahms	Foster	Joplin	Prokofiev	Vecchi
Buxtehude	Frescobaldi	Josquin	Ravel	Victoria
Byrd	Gershwin	Landini	Scarlatti	Vivaldi
				Weber

Genres

Ballate	Etudes	Motets	Scherzos	Symphonies
Ballads	Fugues	Preludes	Sonatas	Virelais
Chorales	Madrigals	Ragtime	Sonatina	Waltzes
Contrafacta	Mazurkas	Quartets		



VerovioHumdrumViewer Scarlatti, Sonata in C minor, L.10, K.84

File View Edit Analysis Scores Help **A** ☐

1 **!!!COM:** Scarlatti, Domenico
2 **!!!CDT:** 1685-1757
3 **!!!OTL:** Sonata in C minor, L.10, K.84
4 **!!!SCT:** K. 84
5 **!!!SCT:** L. 10
6 **!!!OMD:** Allegro ([quarter]=152)
7 ****kern** ****kern** ****dynam**
8 ***staff2** ***staff1** ***staff1/2**
9 ***Icamba** ***Icamba** *****
10 ***I"L.10 (K.84)** ***I"L.10 (K.84)**
11 ***>[A,A1,A,A2,B,B1,B,B2]** ***>[A,A1,A,A2,B,**
12 ***>norep[A,A2,B,B2]** ***>norep[A,A2,B,**
13 ***>A** ***>A** ***>A**
14 ***clefF4** ***clefG2** *****
15 ***k[b-e-a-]** ***k[b-e-a-]** *****
16 ***M3/4** ***M3/4** *****
17 ***MM152** ***MM152** *****
18 **=1-** **=1-** **=1-**
19 **4CC 4C** **8r** **f**
20 **.** **8c'L** **.**
21 **4r** **8e-'** **.**
22 **.** **8g'** **.**
23 **4r** **8cc'** **.**
24 **.** **8ee-'J** **.**
25 **=2** **=2** **=2**
26 **2.r** **(8gg^L** **.**
27 **.** **8ccc)** **.**
28 **.** **8gg'** **.**
29 **.** **8ee-'** **.**
30 **.** **8cc'** **.**
31 **.** **8g'J** **.**
32 **=3** **=3** **=3**
33 **8r** **4c'** **.**
34 **8c'L** **.** **.**

Allegro (♩ = 152)

L.10 (K.84)

f

p

sf p

p

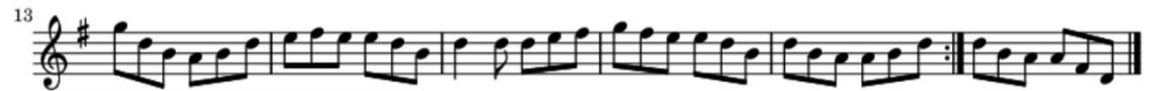
24

ABC

https://en.wikipedia.org/wiki/ABC_notation

The Legacy Jig

jig



▶ 0:54 / 0:54 ——— 🔊 ⋮

```
<score lang="ABC">
```

```
X:1
```

```
T:The Legacy Jig
```

```
M:6/8
```

```
L:1/8
```

```
R:jig
```

```
K:G
```

```
GFG BAB | gfg gab | GFG BAB | d2A AFD |
```

```
GFG BAB | gfg gab | age edB | 1 dBA AFD :| 2 dBA ABd | :
```

```
efe edB | dBA ABd | efe edB | gdB ABd |
```

```
efe edB | d2d def | gfe edB | 1 dBA ABd :| 2 dBA AFD | ]
```

```
</score>
```

ABC

<https://abcnotation.com/examples>

C, D, E, F, | G, A, B, C | D E F G | A B c d | e f g a | b c' d' e' | f' g' a' b' |

A/4 A/2 A/ A A2 A3 A4 A6 A7 A8 A12 A16

A>A A2>A2 | A<A A2<A2 | A>>A A2>>>A2 | A<<A A2<<<A2

(2AB (3ABA (4ABAB (5ABABA (6ABABAB (7ABABABA|

(AA) (A(A)A) ((AA)A) (A|A) A-A A-A-A A2-|A4|

__A _A =A ^A ^^A

"A"A "Gm7"D "Bb"F "F#"A



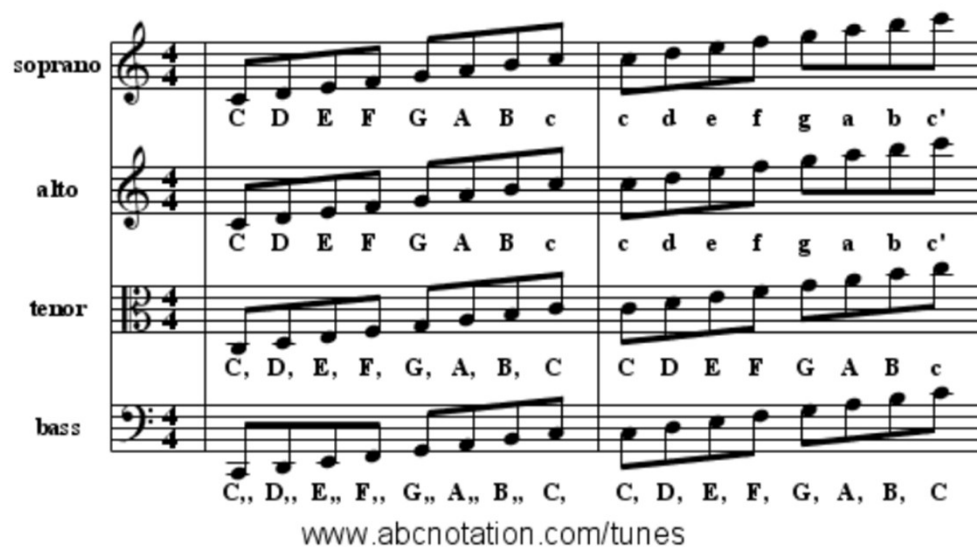
{g}A3 A{g}AA|{gAGAG}A3 {g}A{d}A{e}A|

ABC

https://abcnotation.com/tunePage?a=trillian.mit.edu/~jc/music/abc/src/jcabc2ps-20090401/abc/V_SAT1B2/0000

- Can support multiple voices

TEST: Vocal SATB score.



The image displays a musical score for four voices: soprano, alto, tenor, and bass. The score is written in 4/4 time and consists of two measures. The soprano and alto parts are in treble clef, while the tenor and bass parts are in bass clef. The notes are written on staves, and the lyrics are provided below each staff. The lyrics for the first measure are 'C D E F G A B c' and for the second measure are 'c d e f g a b c'.

www.abcnotation.com/tunes

```
X:1
T: TEST: Vocal SATB score.
N: All parts written "on the staff" with commas.
N: All parts should show scale from C below to above staff.
K: C
V:S name="soprano" clef=treble middle=B
V:A name="alto" clef=treble middle=B
V:T name="tenor" clef=tenor middle=B,
V:B name="bass" clef=bass middle=D,
%
V:S
| CDEF GABc | cdef gabc' |
w: C D E F G A B c | c d e f g a b c' |
V:A
| CDEF GABc | cdef gabc' |
w: C D E F G A B c | c d e f g a b c' |
V:T
| C,D,E,F, G,A,B,C | CDEF GABc |
w: C, D, E, F, G, A, B, C | C D E F G A B c |
V:B
| C,,D,,E,,F,, G,,A,,B,,C, | C,D,E,F, G,A,B,C |
w: C,, D,, E,, F,, G,, A,, B,, C, | C, D, E, F, G, A, B, C |
```

ABC

- A **text-like** representation of **sheet music**
- Readily tokenized
- Support **score** only, not performance
- Generative models adopting the ABC notation: *Folk RNN* [sturm15ismir-lbd]

```
M: 4/4
K: Cmaj
|: G2E>CE>CE>C | G2E>Gc>GE>C | D2D>F (3DED [B,C]>E | C2 (3ECCB>FD>A |
G2E>CE>CE>C | G2E2G>CE>C | D2d2G>Bd>B | 1c4c2 (3GAB:| 2c2B>AC3G/2A/2
|: B>cd>ef>dB>c | d>ed>ec>A (3GAB | c2e>cc4 | e>a (3eee c2 (3GAB |
c2c>ef>ed>c | _B2B2A<Bd<c | B>GF>DD>_BB<d | 1c2B2c>AG<B:| 2c2B2c2c2 |
```



ABC

- A *text-like* representation of **sheet music**
- Readily tokenized
- Support **score** only, not performance
- Generative models adopting the ABC notation
- **Can be used by large language models (LLMs)**



can you generate music?



While I can't directly generate or play music files, I can help create music in **ABC notation** based on your specifications! If you provide details like the key, melody, or chords, I can generate the notation text for you, which you can then input into an ABC music editor (like EasyABC or ABCjs) to see the sheet music and hear the playback.

Would you like me to create something specific?

LLM ♡ ABC

Web-sourced Musical Knowledge

A chord, in music, is any harmonic set of pitches consisting of multiple notes (also called "tones") that are sounded simultaneously ...

Construct smooth-flowing chord progressions for the supplied music.
|: BA | G2 g2^(C)" edeg | B2 BA^(D7)"
BcBA | G2 g2 edeg | dBAG A2 BA | G2 ...

Score Generation Instructions

Training



ChatMusician

Music Theory QA

Which of the following chord progressions best describes the above example?

Composing

Produce sheet music by implementing ABC notation based on the given text.
Rhythm: hornpipe, Key: Gmin, Note Length: 1/8, Meter: 4/4

X:1 L:1/8 Q:1/8=200 M:4/4 K:Gmin |:"Gm" BGdB ...

Chat

Please recommend a song from Taylor Swift.

Sure! One highly recommended song is "Blank Space" from her 2014 album "1989"...

ChatMusician: Understanding and generating music intrinsically with LLM, ACL 2024

<https://github.com/hf-lin/ChatMusician>

Question: "Which of the following statements about triads is correct?",

Options:

A: "A triad can only be composed of three notes.",

B: "All triads are consonant chords.",

C: "When a triad is inverted, its properties and consonance remain consistent with its original position.",

D: "A triad is defined as a chord formed by stacking three notes in a third relationship."},

Answer: D

Library: EasyABC

<https://github.com/jwdj/EasyABC/>

EasyABC / midi2abc.py

Features

- Good ABC standard coverage thanks to internal use of abcm2ps and abc2midi
- Syntax highlighting
- Zoom support
- Import MusicXML, MIDI and Noteworthy Composer files.
- Export to MIDI, SVG, PDF (single tune or whole tune book).
- Select notes by clicking on them and add music symbols by using drop-down menus in the toolbar.
- Play the active tune as midi (using SoundFont)
- See which notes are currently playing (Follow score)

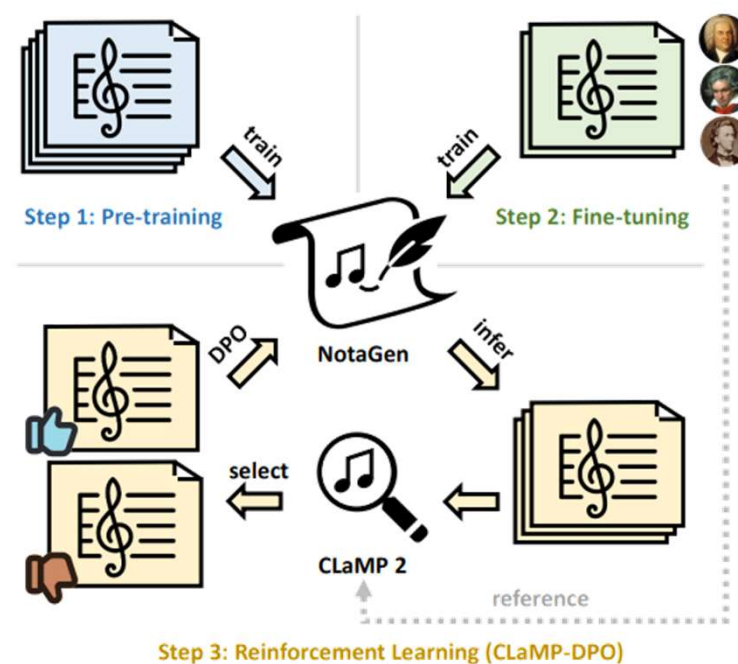
NotaGen

<https://electricalexis.github.io/notagen-demo/>

- Pre-trained on 1.6M pieces of music in **ABC** notation
- Fine-tuned on approximately 9K high-quality classical compositions
- Post-trained with **CLaMP-DPO**

Data Sources	Amount
<i>DCML Corpora</i>	560
<i>OpenScore String Quartet Corpus</i>	342
<i>OpenScore Lieder Corpus</i>	1,334
<i>ATEPP</i>	55
<i>KernScores</i>	221
<i>Internal Sources</i>	6,436
<i>Total</i>	8,948

Table 1: Data sources and the respective amounts for fine-tuning.

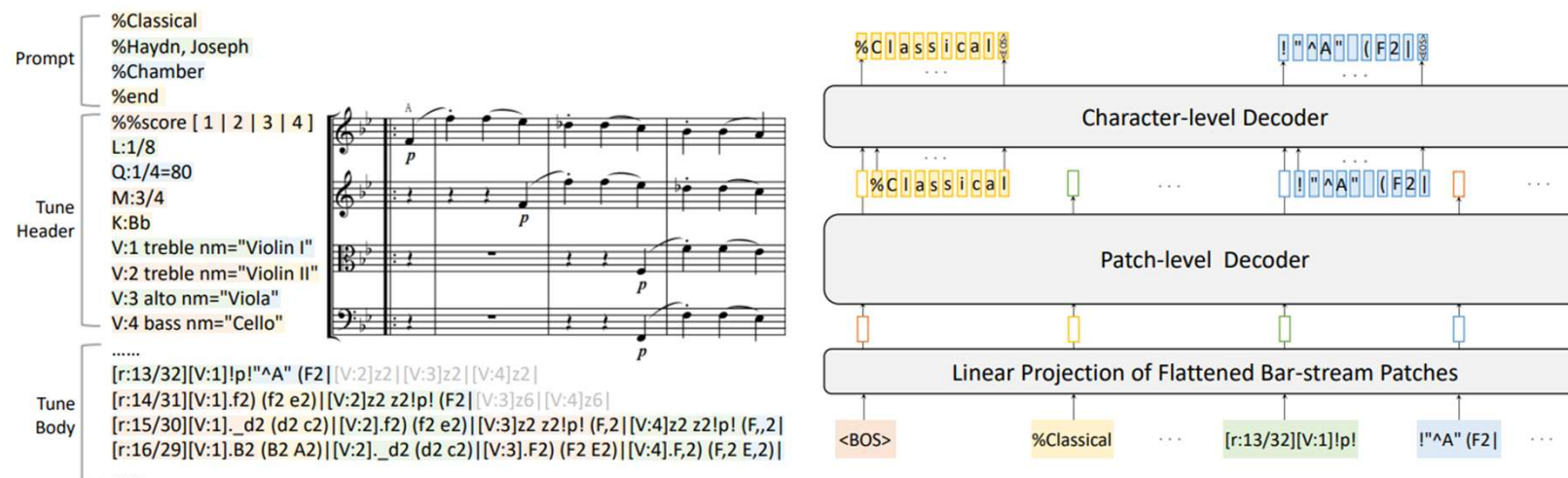


Ref: Wang et al, “NotaGen: Advancing musicality in symbolic music generation with large language model training paradigms,” IJCAI 2025

NotaGen

<https://electriclexis.github.io/notagen-demo/>

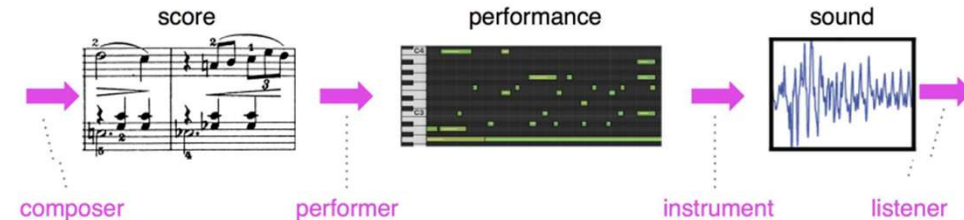
- Pre-trained on 1.6M pieces of music in **ABC** notation
- Fine-tuned on approximately 9K high-quality classical compositions
- Post-trained with **CLaMP-DPO**



Ref: Wang et al, “NotaGen: Advancing musicality in symbolic music generation with large language model training paradigms,” IJCAI 2025

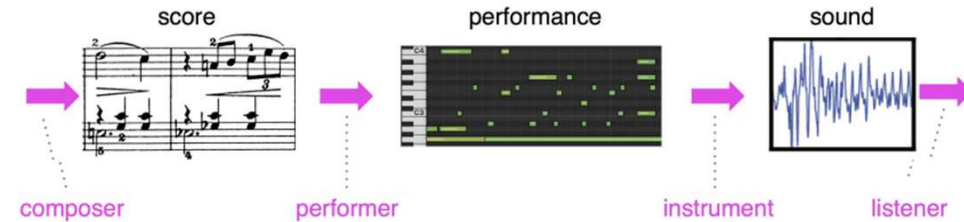
Summary

- **Sheet Music / MusicXML / ABC**
 - Support **score** only, not performance
 - Use **musical time** only, not physical time
 - Richer information about the score itself
- **Piano roll / MIDI**
 - Support **both score and performance**
 - Use either **musical time** (after quantization) or **physical time**
 - Rich performance information
 - Datasets more easily accessible and thereby **more widely used**



Summary (Cont')

- **ABC**
 - Good for simple melodies, chords, and basic arrangements
 - Limited for more complicated scores (e.g., piano, symphony)
 - Good for LLMs (but can *only* do composition; not performance)
- **Piano roll / MIDI / MusicXML**
 - More flexible



ChatGPT-6 Astra

<https://x.com/aug5thmusic/status/2096030719156089029>

- “GPT-6 Astra has the best result yet on the Bach Benchmark”
- Prompt below
 - “In LilyPond (version 2.24), write a 4-part chorale in the style of Bach, 3/4 time, G minor. Use two staves -- soprano and alto on the top staff, tenor and bass on the bottom staff. Respond with only the code block.”

ChatGPT-6 Astra Chorale in G Minor

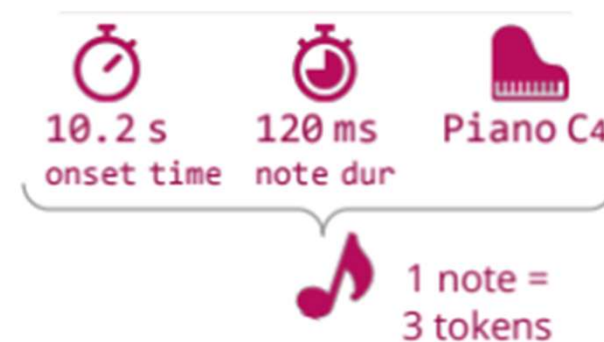


MIDI + LLM?

- MIDI tokens are “artificial,” no standard way of tokenization
- How if we just fine-tune an LLM to expand the vocabulary to include a certain set of MIDI tokens?

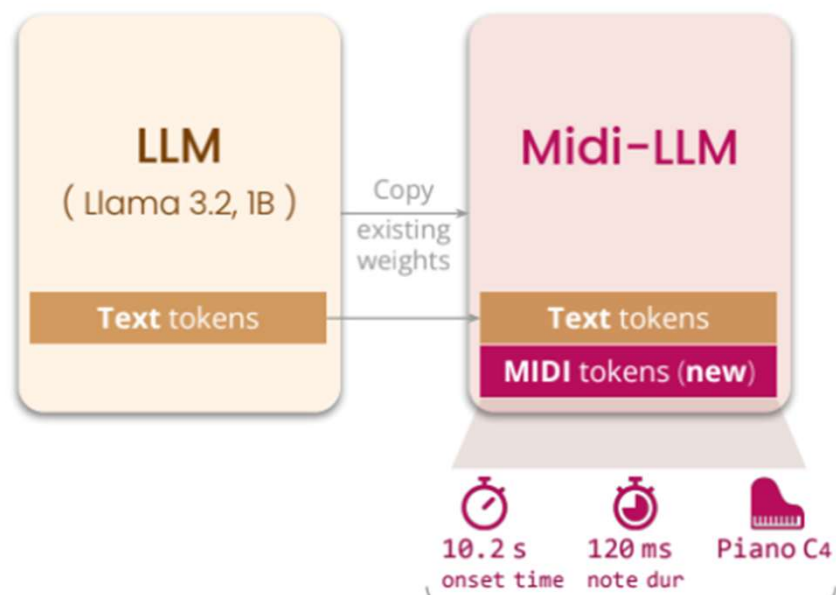
Token Design in Multi-track MIDI Transformer

- To further account for instruments
- The tokenization used in MIDI-LLM (Wu et al 2025)
 - **Arrival (onset) time**: note's start time (0th to 100th second with 10 ms quantization)
 - **Note duration**: How long the note is held (0 to 10 seconds with 10 ms quantization)
 - **Instrument-pitch**: A joint token for the instrument and its pitch (129 MIDI inst. × 128 pitches)
- This amounts to **27.5K** possible tokens



MIDI-LLM: An LLM for Generating Multitrack MIDI Music

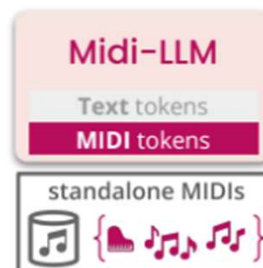
<https://midi-llm-demo.vercel.app/>



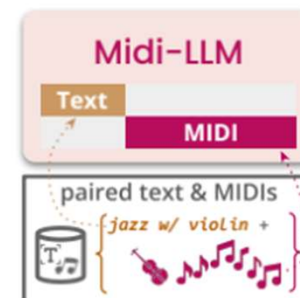
$$\mathbf{E}_{\text{MIDI-LLM}} := \begin{bmatrix} \mathbf{E}_{\text{LLM}}^T & \mathbf{E}_{\text{AMT}}^T \end{bmatrix}^T \in \mathbb{R}^{(|\mathcal{V}_{\text{LLM}}| + |\mathcal{V}_{\text{AMT}}|) \times D}$$

Training – 2-Stage Pipeline

Stage #1
Continued pretraining



Stage #2
Text-to-MIDI finetuning



Wu et al, “MIDI-LLM: Adapting large language models for text-to-MIDI music generation”, AI4Music, 2025

MIDI-LLM: An LLM for Generating Multitrack MIDI Music

Table 1: MIDI-LLM two-stage training data summary.

		<i>Continued Pretraining</i>		<i>Finetuning</i>
		Text	MIDI	Text -to- MIDI
Domain(s)	• Music-related web articles (56%) • Synthetic music knowledge (by GPT-4) (17%) • QAs – music in ABC notation (16%) • QAs – common knowledge (12%)	Standalone multitrack MIDI	Multitrack MIDI	paired w/ text
Dataset(s)	(Subset of) MusicPile (Yuan et al., 2024)	GigaMIDI (Lee et al., 2025)	MidiCaps (for paired text) (Melechovsky et al., 2024) + LMD (for MIDI content) (Raffel, 2016)	
# Tokens	1.69 B	1.38 B	1.71 B (w/o augmentation)	
SeqLen	2048	2048 ($\approx$ 30 sec. music)	Text $\leq$ 256 (as prefix) MIDI 2048	

Wu et al, “MIDI-LLM: Adapting large language models for text-to-MIDI music generation”, AI4Music, 2025

Appendix: html-midi-player

<https://cifkao.github.io/html-midi-player/>

Getting started

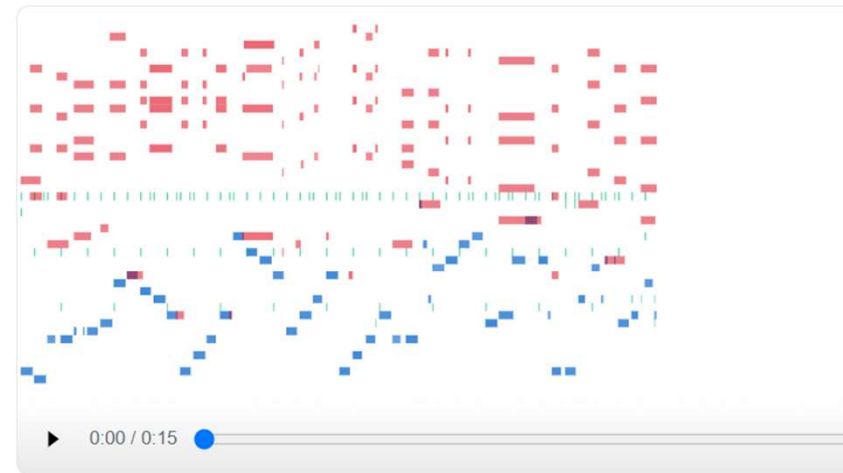
1. Add the necessary scripts to your page:

```
<script src="https://cdn.jsdelivr.net/combine/npm/tone@14.7.58,npm/@magenta/music@1.23.1/es6/core...">
```

2. Add a player and (optionally) a visualizer:

```
<midi-player  
  src="https://magenta.github.io/magenta-js/music/demos/melody.mid"  
  sound-font visualizer="#myVisualizer">  
</midi-player>  
<midi-visualizer type="piano-roll" id="myVisualizer"></midi-visualizer>
```

That's it!



Appendix: midi-realplayer

<https://midirealplayer.vaclis.net/demo/>

